

Data Mining and Analytics (CSDC - 0337)

Report

Title:

Comprehensive Analysis of Student Academic and Behavioral Dataset using Python and Data Visualization Techniques

Abstract:

This project presents a comprehensive analysis of student academic and behavioral data using Python-based data analytics and visualization tools.

- The **main aim** of the study is to identify the key factors influencing students' CGPA, academic progress, and satisfaction levels with their college experience. By analyzing data related to school activities, stream selection, reasons affecting performance, and personal interests, the project provides deep insights into how various academic and non-academic elements shape student outcomes.

- The analysis was conducted using a combination of **Python libraries**, including **Pandas** for data cleaning and manipulation, **Matplotlib** and **Seaborn** for data visualization, and **NumPy** for numerical processing. The dataset underwent multiple stages of preprocessing—such as removal of irrelevant or inconsistent entries, renaming and standardizing column names, imputing missing values, and categorizing numerical data into meaningful ranges.

Through step-by-step visual exploration, the study examined correlations between academic grades (10th, 12th, CGPA), participation in extracurricular activities, stream choice motivation, and overall college satisfaction.

- Key insights revealed that students who selected their academic streams based on **personal interest** and actively engaged in **academic or technical activities** showed higher CGPA scores and greater satisfaction with college life. Conversely, time-management issues, peer influence, and lack of interest in the chosen branch were identified as major reasons behind declining academic performance.

- The **outcome** of the project is a detailed visual report that not only highlights patterns and relationships within the dataset but also serves as a decision-support resource for educators and institutions. The findings can help colleges develop targeted strategies to improve student engagement, performance, and well-being.

Introduction:

The primary goal of this project is to analyze and understand student performance trends and behavioral patterns using real academic data. The dataset used for this study was obtained from a **CSV file** containing detailed responses from students regarding their academic records, extracurricular activities, stream selection choices, motivations, satisfaction levels, and other influencing factors such as time management, health, and interest in studies.

The project focuses on uncovering **how various personal, academic, and social elements contribute to a student's overall performance and satisfaction** in college. Through step-by-step analysis, it aims to answer key questions such as — *How do school activities influence CGPA? Does choosing a stream based on personal interest lead to better results? What are the most common reasons behind academic improvement or decline?*

This type of analysis is extremely valuable for **educational institutions**, as it provides data-driven insights into student behavior and academic trends. Administrators and faculty can use the findings to develop better mentoring programs, enhance learning environments, and design interventions that address specific challenges faced by students. Additionally, understanding the relationship between satisfaction levels and academic outcomes can help improve the overall student experience and institutional reputation.

The project also emphasizes **data preprocessing and cleaning**, as the dataset initially contained inconsistencies, missing values, and unstandardized text entries. By performing transformations such as renaming columns, imputing categorical variables, and filtering irrelevant rows, a refined and accurate dataset was achieved for reliable analysis. Subsequently, multiple visualizations—such as pie charts, bar graphs, scatter plots, and heatmaps—were generated to uncover patterns and correlations between academic and behavioral variables.

In summary, this study provides a **comprehensive analytical journey**, starting from raw data cleaning to generating actionable insights, thus bridging the gap between raw information and educational decision-making.

Tools and Libraries Used:

The analysis was performed using the **Python programming language**, with a focus on powerful data analysis and visualization libraries. The key tools used in this project are as follows:

1. **Pandas** – Used for **data manipulation and cleaning**. It helped in reading the CSV file, handling missing values, transforming data into usable formats, and performing aggregations like grouping and summing categorical data.
2. **NumPy** – Provided **numerical operations** and efficient handling of arrays, particularly when categorizing CGPA ranges or computing derived numerical values.
3. **Matplotlib** – Used for **creating basic visualizations** such as pie charts and bar plots that display categorical data distributions clearly.

4. **Seaborn** – A statistical visualization library built on top of Matplotlib, used for **advanced visualizations** such as boxplots, scatter plots, and heatmaps to show relationships between multiple variables with appealing aesthetics.
5. **Warnings** – Utilized to **suppress unnecessary warning messages** during execution to ensure clean output during iterative testing and visualization.
6. **Jupyter Notebook / Google Colab** – These served as the **execution environments** for running Python code interactively, displaying data outputs, and visualizations in real-time, which enhanced the exploratory data analysis process.

Data Import and Initial Exploration:

Steps covered:

- Imported the CSV using `pd.read_csv()`
- Displayed first few rows using `df.head()`
- Used `df.info()` and `df.describe()` to understand structure and data types
- Checked missing values with `df.isnull().sum()`

```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/Acad-Data.csv', encoding='latin-1')
display(df.head())

df.info()
df.describe()
df.isnull().sum()
```

	Name	Year of Study	Branch	What type of area was your school located in?	What was your percentage/cgpa in Class 10th?	What was your percentage/cgpa in Class 12th?	Were you involved in any activities during school (Class 9th&10th&12th)?	How did you choose your stream after Class 10th?	What is your current CGPA?	What was CG pre Year/Sem
0	Mahi	4th Year	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2	
1	Bhav Na Kha	3rd Year	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7	
2	guri69	3rd Year	ice	Semi-urban	Above 90%	Between 80% - 90%	Sports (Music/Dance/Drama, etc.)...	Own interest	7.7	
3	Falguni2002	4th Year	CE	Urban	Between 60% - 80%	Between 60% - 80%	None	Own interest	6.3	
4	Plaksha Kapoor	3rd Year	IPE	Semi-urban	Above 90%	Above 90%	Sports	Own interest	7.79	7.0(1s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 137 entries, 0 to 136
Data columns (total 18 columns):
 #   Column                                                                                               Non-Null Count  Dtype
---  -
 0   Name :                                                                                               137 non-null    object
Note: If you don't want to share your real name, just use a fun/unique name like Jacky53, Munja89, etc. Please don't leave it blank with random stuff or bad words keep it decent ones
 1   Year of Study                                                                                       137 non-null    object
 2   Branch                                                                                               137 non-null    object

Example: CSE, ME, CE...etc
 3   What type of area was your school located in?                                                    137 non-null    object
 4   What was your percentage/cgpa in Class 10th?                                                       137 non-null    object
 5   What was your percentage/cgpa in Class 12th?                                                       137 non-null    object
 6   Were you involved in any activities during school (Class 9th&12th)?                               137 non-null    object
 7   How did you choose your stream after Class 10th?                                                  137 non-null    object
 8   What is your current CGPA?                                                                           136 non-null    object
 9   What was your CGPA in previous Year/Semester?                                                     137 non-null    object

1st yr: As per Semester
Other yrs: As per Year
10  How has your CGPA changed over time?                                                                135 non-null    object
11  What are the main reasons affecting your CGPA? (You can select multiple)                            137 non-null    object
12  Rate our college's involvement in your personal & academic growth?                                137 non-null    int64
13  Are you pursuing any side interests or passions in college?                                        137 non-null    object
14  If above you selected "Yes". Please mention.                                                         83 non-null    object
15  What is your main motivation for studying in college?                                              137 non-null    object
16  Are you satisfied with your academic journey so far?                                              137 non-null    object
17  Gender                                                                                               137 non-null    object
dtypes: int64(1), object(17)
memory usage: 19.4+ KB
```

Name :	Note: If you don't want to share your real name, just use a fun/unique name like Jacky53, Munja89, etc. Please don't leave it blank with random stuff or bad words keep it decent ones	0
Year of Study		0
Branch	Example: CSE, ME, CE...etc	0
What type of area was your school located in?		0
What was your percentage/cgpa in Class 10th?		0
What was your percentage/cgpa in Class 12th?		0
Were you involved in any activities during school (Class 9th&12th)?		0
How did you choose your stream after Class 10th?		0
What is your current CGPA?		1
What was your CGPA in previous Year/Semester?	1st yr: As per Semester Other yrs: As per Year	2
How has your CGPA changed over time?		0
What are the main reasons affecting your CGPA? (You can select multiple)		0
Rate our college's involvement in your personal & academic growth?		0
Are you pursuing any side interests or passions in college?		0
If above you selected "Yes". Please mention.		54
What is your main motivation for studying in college?		0
Are you satisfied with your academic journey so far?		0
Gender		0

Data Cleaning and Preprocessing:

(a) Removing Bad / Irrelevant Text

```
import pandas as pd

bad_words = ['backshot', 'haven't given exams', 'hale', 'i'm', 'papers', 'fresher']
df = df[~df.apply(lambda row: row.astype(str)
                  .str.lower()
                  .apply(lambda x: any(bad_word.lower() in x for bad_word in bad_words))
                  .any(), axis=1)]

df.reset_index(drop=True, inplace=True)

df.head()
```

Name : Note: If you don't want to share your real name, just use a fun/unique name like Jacky53, Munja89, etc. Please don't leave it blank with random stuff or bad words keep it decent													Rate our college's involvement in your personal & academic growth?
	Year of Study	Branch Example: CSE, ME, CE...etc	What type of area was your school located in?	What was your percentage/cgpa in Class 10th?	What was your percentage/cgpa in Class 12th?	Were you involved in any activities during school (Class 9th-12th)?	How did you choose your stream after Class 10th?	What is your current CGPA?	What was your CGPA in previous Year/Semester? 1st yr: As per Semester Other yrs: As per Year	How has your CGPA changed over time?	What are the main reasons affecting your CGPA? (You can select multiple)		
0	Mahi	4th Year	Urban	Between 80% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2	7.5	Declined	Focus on other passions/projects		
1	Bhav Na Kha	3rd Year	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7	8	Declined	Time management issues		
2	guni89	3rd Year	ice	Semi-urban	Above 90%	Sports, Arts (Music/Dance/Drama, etc.)...	Own interest	7.7	7.5	Mix (up and down every semester)	Attendance rules, Focus on other passions...		
3	Falguni2002	4th Year	CE	Urban	Between 80% - 80%	None	Own interest	6.3	6.6	Declined	Friends / Peer influence		
4	Plaksha Kapoor	3rd Year	IPE	Semi-urban	Above 90%	Sports	Own interest	7.79	7.0(1st year)	Improved	Time management issues		

(b) Renaming Columns

```
new_column_names = {
    'Name': 'Name',
    'Year of Study': 'Year',
    'Branch': 'Branch',
    'What type of area was your school located in?': 'School_Area',
    'What was your percentage/cgpa in Class 10th?': '10th',
    'What was your percentage/cgpa in Class 12th?': '12th',
    'Were you involved in any activities during school (Class 9th-12th)?': 'School_Activities',
    'How did you choose your stream after Class 10th?': 'Stream_Choice',
    'What is your current CGPA?': 'Current_CGPA',
    'What was your CGPA in previous Year/Semester? 1st yr: As per Semester\nOther yrs: As per Year': 'Previous_CGPA',
    'How has your CGPA changed over time?': 'CGPA_Change',
    'What are the main reasons affecting your CGPA? (You can select multiple)': 'CGPA_Reasons',
    'Rate our college's involvement in your personal & academic growth?': 'College_Support',
    'Are you pursuing any side interests or passions in college?': 'Side_Interests',
    'If above you selected "Yes". Please mention.': 'Side_Interests_Details',
    'What is your main motivation for studying in college?': 'College_Motivation',
}
```

```
'Are you satisfied with your academic journey so far?': 'Academic_Satisfaction',
'Gender ': 'Gender'
}

df.columns.values[0] = 'Name'
df.columns.values[6] = 'School_Activities'
df.columns.values[12] = 'Rate our College'

df = df.rename(columns=new_column_names)
display(df.head())
```

	Name	Year	Branch	School_Area	10th	12th	School_Activities	S
0	Mahi	4th Year	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	
1	Bhav Na Kha	3rd Year	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	

Stream_Choice	Current_CGPA	Previous_CGPA	CGPA_Change	CGPA_Reasons	Rate our College	Side_Interests	Side_Interest
---------------	--------------	---------------	-------------	--------------	------------------	----------------	---------------

College_Motivation	Academic_Satisfaction	Gender
Both	Yes	Male
Knowledge & Learning	Yes	Female
Both	No	Male
Both	Maybe	Male
Both	Maybe	Female

(c) Columns Processing

```
df['Year'] = df['Year'].astype(str).str.extract(r'(\d+)').astype(float).astype('Int64')
display(df.head())
```

Explanation

- Converts the 'Year' column to string to ensure text processing works correctly.
- Uses regular expression (\d+) to extract only the numeric part (year) from the text.
- Converts the extracted values to float, then to integer (Int64) for numerical consistency.
- Displays the first few cleaned rows to confirm successful conversion.

	Name	Year	Branch	School_Area	10th	12th	School_Activities	Stream_Choice	Current_CGPA
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7
2	guri69	3	ice	Semi-urban	Above 90%	Between 80% - 90%	Sports, Arts (Music/Dance/Drama, etc.)...	Own interest	7.7
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	None	Own interest	6.3
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	Sports	Own interest	7.79

```
df['School_Area'] = df['School_Area'].str.replace(r'^a-zA-Z\s-', '', regex=True).str.strip()
display(df.head())
```

Explanation:

- Removes all unwanted symbols, numbers, or punctuation from the 'School_Area' column using a regular expression.
- Keeps only alphabets, spaces, and hyphens (a-zA-Z\s-).
- str.strip() removes extra spaces from the beginning and end of the text.
- Displays the cleaned output for verification.

	Name	Year	Branch	School_Area	10th	12th	School_Activities	Stream_Choice	Current_CGPA	Previous_CGPA	CGPA_Change	CGPA_Reasons
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2	7.5	Declined	Focus on other passions/projects
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7	8	Declined	Time management issues
2	guri69	3	ice	Semi-urban	Above 90%	Between 80% - 90%	Sports, Arts (Music/Dance/Drama, etc.)...	Own interest	7.7	7.5	Mix (up and down every semester)	Attendance rules, Focus on other passions...
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	None	Own interest	6.3	6.6	Declined	Friends / Peer influence
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	Sports	Own interest	7.79	7.0(1st year)	Improved	Time management issues

```
df = df.dropna(how='all')
display(df.head())
```

Explanation:

- Drops rows where **all columns** are NaN (completely empty rows).
- This ensures the dataset only contains meaningful data.
- Displays the resulting DataFrame after removing empty rows.

	Name	Year	Branch	School_Area	10th	12th	School_Activities	Stream_Choice	Current_CGPA	Previous_CGPA	CGPA_Change	CGPA_Reasons
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2	7.5	Declined	Focus on other passions/projects
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7	8	Declined	Time management issues
2	guri69	3	ice	Semi-urban	Above 90%	Between 80% - 90%	Sports, Arts (Music/Dance/Drama, etc.)...	Own interest	7.7	7.5	Mix (up and down every semester)	Attendance rules, Focus on other passions...
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	None	Own interest	6.3	6.6	Declined	Friends / Peer influence
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	Sports	Own interest	7.79	7.0(1st year)	Improved	Time management issues

(d)Branch Column

```
df['Branch'] = df['Branch'].astype(str).str.strip().str.upper().replace('DSE', 'DS')
unique_branch_values = df['Branch'].unique()
print("Unique values in the 'Branch' column:")
for branch in unique_branch_values:
    print(branch)
```

```
df['Branch'] = df['Branch'].astype(str).str.strip().str.upper().replace('DSE', 'DS')

valid_branches = ['CSE', 'ECE', 'IT', 'ME', 'EE', 'CE', 'CH', 'BT', 'IPE', 'ICE', 'DS', 'MNC',
'MCE']

manual_corrections = {
    'YOUR FATHERS BRANCH': 'CSE',
    'MCE OR MNC': 'MNC',
    "": None,
}
a
df['Branch'] = df['Branch'].replace(manual_corrections)

df.loc[~df['Branch'].isin(valid_branches), 'Branch'] = 'UNKNOWN'

unique_branch_values = df['Branch'].unique()
print("✅ Cleaned unique values in the 'Branch' column:")
for branch in unique_branch_values:
    print(branch)
```

Explanation

- This code focuses on cleaning and standardizing the 'Branch' column in the dataset.
- First, all entries are converted to uppercase and extra spaces removed using `str.strip()` and `str.upper()` to ensure uniform formatting.
- The abbreviation 'DSE' is replaced with 'DS' for consistency.
- Next, a list of valid branch names is defined to serve as a reference for data validation.
- Some specific wrong or unclear entries like "YOUR FATHERS BRANCH" or "MCE OR MNC" are manually corrected using a dictionary called `manual_corrections`.
- Any branch names not in the valid list are replaced with 'UNKNOWN' to maintain dataset consistency.
- Finally, unique values are printed both before and after cleaning to verify that all invalid or inconsistent entries were successfully handled.

Unique values in the 'Branch' column:

ECE
IT
ICE
CE
IPE
CSE
BT
EE
ME
CH
DS

✓ Cleaned unique values in the 'Branch' column:

ECE
IT
ICE
CE
IPE
CSE
BT
EE
ME
CH
DS

(e) Data Categorization:

Categorize the '10th' and '12th' columns into grades based on percentage ranges and visualize the distribution of these grades.

```
percentage_ranges = {
    'Above 90%': '>90%',
    'Between 80% - 90%': '80-90%',
    'Between 60% - 80%': '60-80%',
    'Under 60%': '<60%'
}

print("Percentage/CGPA ranges for categorization:")
for key, value in percentage_ranges.items():
    print(f'- {key}: {value}')
```

```
Percentage/CGPA ranges for categorization:
- Above 90%: >90%
- Between 80% - 90%: 80-90%
- Between 60% - 80%: 60-80%
- Under 60%: <60%
```

```
df['10th_grade'] = df['10th'].map(percentage_ranges).fillna('Unknown')
df['12th_grade'] = df['12th'].map(percentage_ranges).fillna('Unknown')
display(df.head())
```

	Name	Year	Branch	School_Area	10th	12th	School_Activities	Stream_Choice	Current_CGPA	Previous_CGPA	CGPA_Change
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	Coding / Tech Competitions	Own interest	7.2	7.5	Decrease
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	Arts (Music/Dance/Drama, etc.)	Own interest	7.7	8	Decrease
2	guri69	3	ICE	Semi-urban	Above 90%	Between 80% - 90%	Sports, Arts (Music/Dance/Drama, etc.)...	Own interest	7.7	7.5	Mix (up/down/semi)
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	None	Own interest	6.3	6.6	Decrease
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	Sports	Own interest	7.79	7.0(1st year)	Improvement

Explanation

- In this step, the dataset's **10th and 12th-grade percentage columns** are **standardized into categories** for easier analysis and visualization.
- A dictionary named `percentage_ranges` is created to **map various textual representations** of marks into **four defined categories**:
 - `>90%` → Excellent performers
 - `80–90%` → Very good performers
 - `60–80%` → Average to good performers
 - `<60%` → Below-average performers
- The `.map()` function is used to **transform** the raw textual data in '10th' and '12th' columns according to the mapping dictionary.
- Any entry not matching the dictionary (e.g., missing or unexpected values) is replaced with 'Unknown' using `.fillna('Unknown')`.
- Finally, the cleaned and categorized columns '10th_grade' and '12th_grade' are displayed to verify successful transformation.

Purpose / Outcome:

This process ensures that all academic performance data is grouped into consistent, interpretable categories.

Such categorization helps in:

- Creating clear visualizations of performance distribution.
- Identifying patterns between school-level performance and later academic or behavioral factors.
- Simplifying statistical and correlation analysis by reducing text variation.

```
tenth_grade_distribution = df['10th_grade'].value_counts(normalize=True)
twelfth_grade_distribution = df['12th_grade'].value_counts(normalize=True)

print("Distribution of 10th Grade:")
display(tenth_grade_distribution)

print("\nDistribution of 12th Grade:")
display(twelfth_grade_distribution)
```

Distribution of 10th Grade:

proportion

10th_grade

>90% 0.372093

80-90% 0.271318

60-80% 0.248062

<60% 0.108527

dtype: float64

Distribution of 12th Grade:

proportion

12th_grade

>90% 0.348837

80-90% 0.341085

60-80% 0.271318

<60% 0.038760

dtype: float64

Explanation

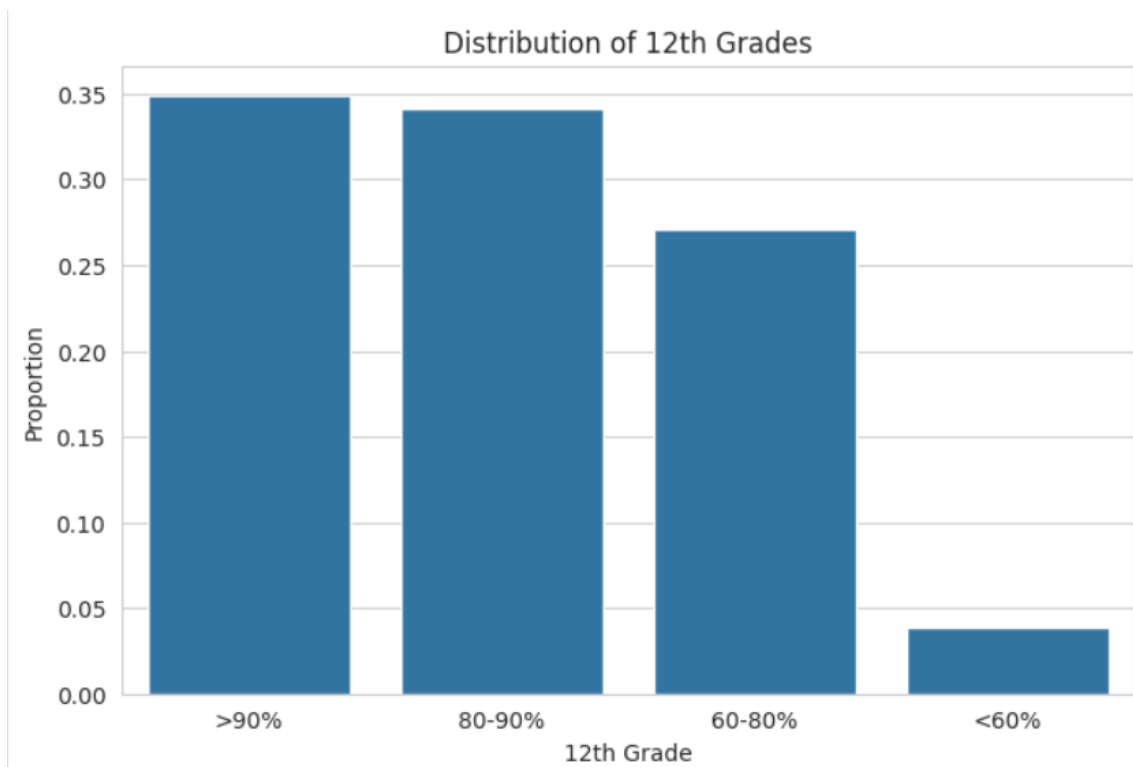
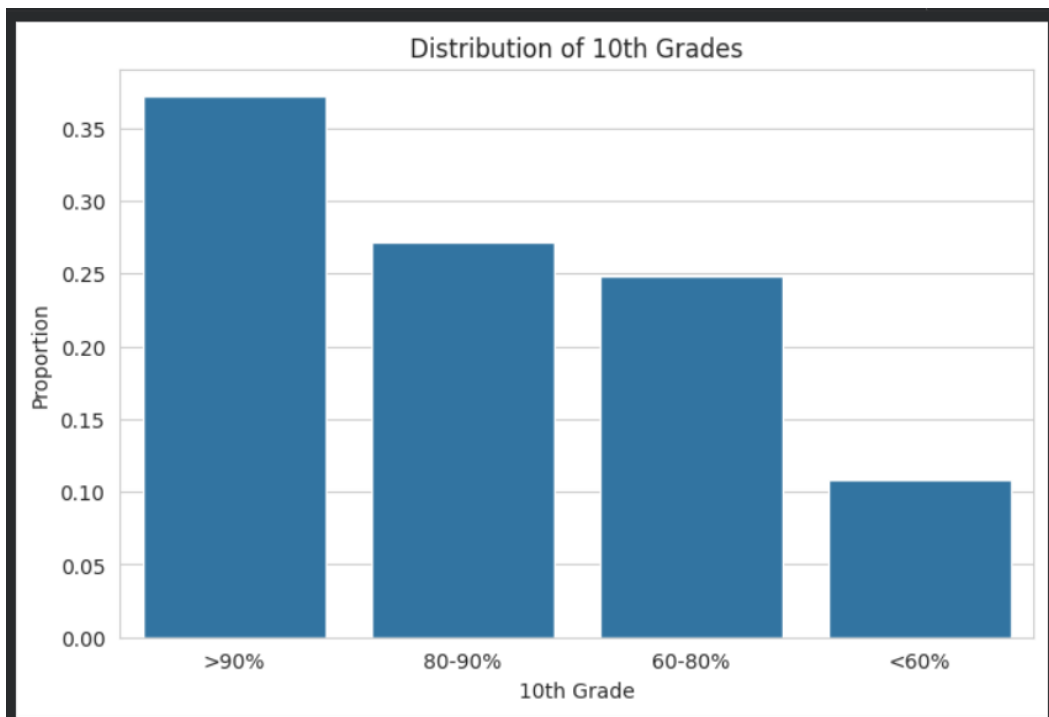
- After categorizing students based on their 10th and 12th grade performance, the next step is to analyze the distribution of these categories.
- The `.value_counts(normalize=True)` method is used here:
 - It counts the frequency of each grade category.
 - By setting `normalize=True`, it converts those counts into percentages (proportional values).
- This helps to understand what percentage of students fall into each performance range (e.g., >90%, 80–90%, etc.).
- Two separate variables are created:
 - `tenth_grade_distribution` — stores the distribution for 10th-grade categories.
 - `twelfth_grade_distribution` — stores the distribution for 12th-grade categories.
- The `display()` function is used to show these results clearly in a tabular form, especially useful in Jupyter Notebook or Google Colab environments.

Visualize it

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.barplot(x=tenth_grade_distribution.index, y=tenth_grade_distribution.values)
plt.xlabel('10th Grade')
plt.ylabel('Proportion')
plt.title('Distribution of 10th Grades')
plt.show()

plt.figure(figsize=(8, 5))
sns.barplot(x=twelfth_grade_distribution.index, y=twelfth_grade_distribution.values)
plt.xlabel('12th Grade')
plt.ylabel('Proportion')
plt.title('Distribution of 12th Grades')
plt.show()
```



- **School Activities Cleaning and Categorization**

```
# Step 1: Checking unique values in the column
unique_school_activities = df['School_Activities'].unique()
print("Unique values in the 'School_Activities' column:")
for activity in unique_school_activities:
    print(activity)

# Step 2: Cleaning function
def clean_activity(text):
    text = str(text).lower()
    text = unidecode.unidecode(text)
    text = re.sub(r'[^w\s,]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Step 3: Apply cleaning
df['School_Activities'] = df['School_Activities'].apply(clean_activity)

# Step 4: Categorize into activity types
options = {
    'Sports': ['sports'],
    'Arts': ['arts', 'arts music/dance/drama etc'],
    'Tech': ['coding / tech competitions', 'tech'],
    'Academics': ['olympiads / quiz / other academics', 'academics'],
    'None': ['none']
}

for col, keywords in options.items():
    df[col] = df['School_Activities'].apply(lambda x: 1 if any(k in x for k in keywords) else 0)

# Step 5: Drop original text column
df.drop(columns=['School_Activities'], inplace=True)

df.head(20)
```

Explanation

This section focuses on extracting and structuring student co-curricular activity data from the dataset.

The goal is to identify each student's participation area (e.g., sports, arts, technology, academics, or none) to study how these factors relate to their academic and satisfaction metrics later.

Here's how the process works step by step:

1. Identify Unique Entries:

- We first print out all unique values in the 'School_Activities' column.
- This helps us understand the variety and inconsistency in how students entered their activity information (e.g., “sports”, “Sports”, “coding / tech competitions”, “arts music/dance/drama etc”).

2. Clean the Text Data:

- A function `clean_activity()` was created to make entries uniform by:
 - Converting all text to lowercase.
 - Removing accents or special characters using `unidecode`.
 - Deleting punctuations and extra spaces with regular expressions (`re`).
- This ensures all entries are clean and standardized before categorization.

3. Categorization into Defined Groups:

- We created a dictionary options that maps each broad activity category to a list of keywords that identify it.
 - Example:
 - “sports” → *Sports* category
 - “coding / tech competitions” → *Tech* category
 - “arts music/dance/drama etc” → *Arts* category
- Then, using `apply()` with a lambda function, a new binary column (0 or 1) is created for each activity type.
 - 1 means the student participated in that category.
 - 0 means they didn't.

4. Drop Redundant Column:

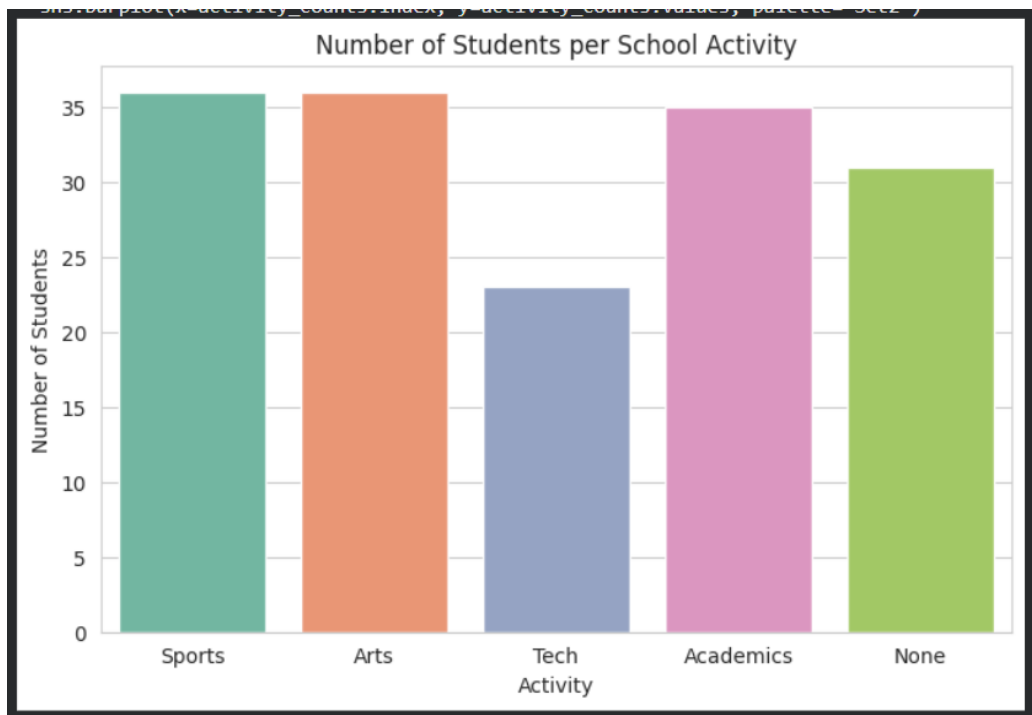
- After categorization, the original 'School_Activities' column is dropped to keep the dataset clean and structured.

Visualise it:

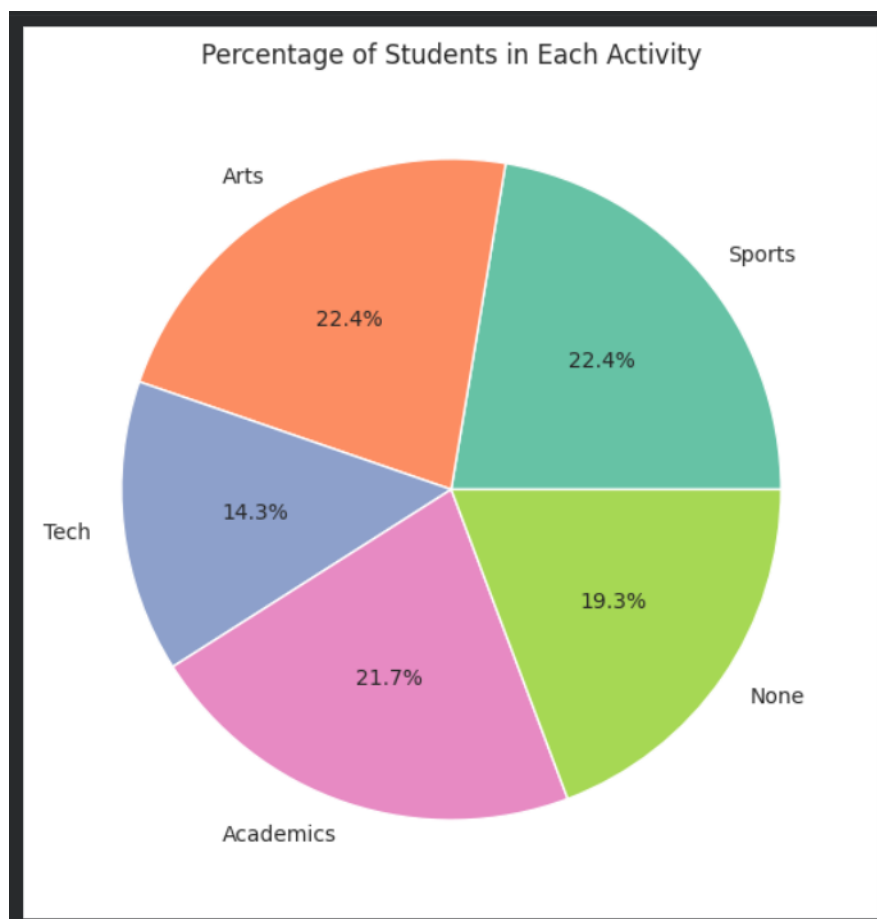
```
import matplotlib.pyplot as plt
import seaborn as sns

activity_counts = df[['Sports', 'Arts', 'Tech', 'Academics', 'None']].sum()

plt.figure(figsize=(8,5))
sns.barplot(x=activity_counts.index, y=activity_counts.values, palette='Set2')
plt.title('Number of Students per School Activity')
plt.ylabel('Number of Students')
plt.xlabel('Activity')
plt.show()
```

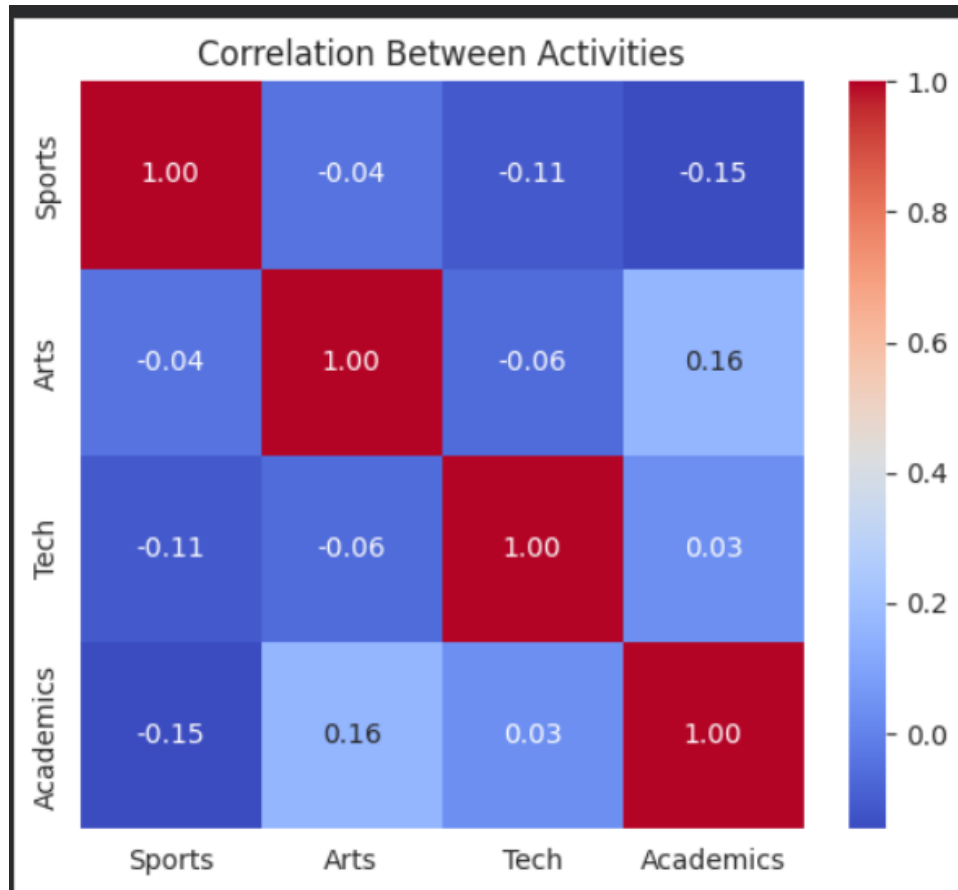



```
plt.figure(figsize=(7,7))
plt.pie(activity_counts.values, labels=activity_counts.index, autopct='%1.1f%%',
        colors=sns.color_palette('Set2'))
plt.title('Percentage of Students in Each Activity')
plt.show()
```



```
# Correlation between activities
corr = df[['Sports','Arts','Tech','Academics']].corr()

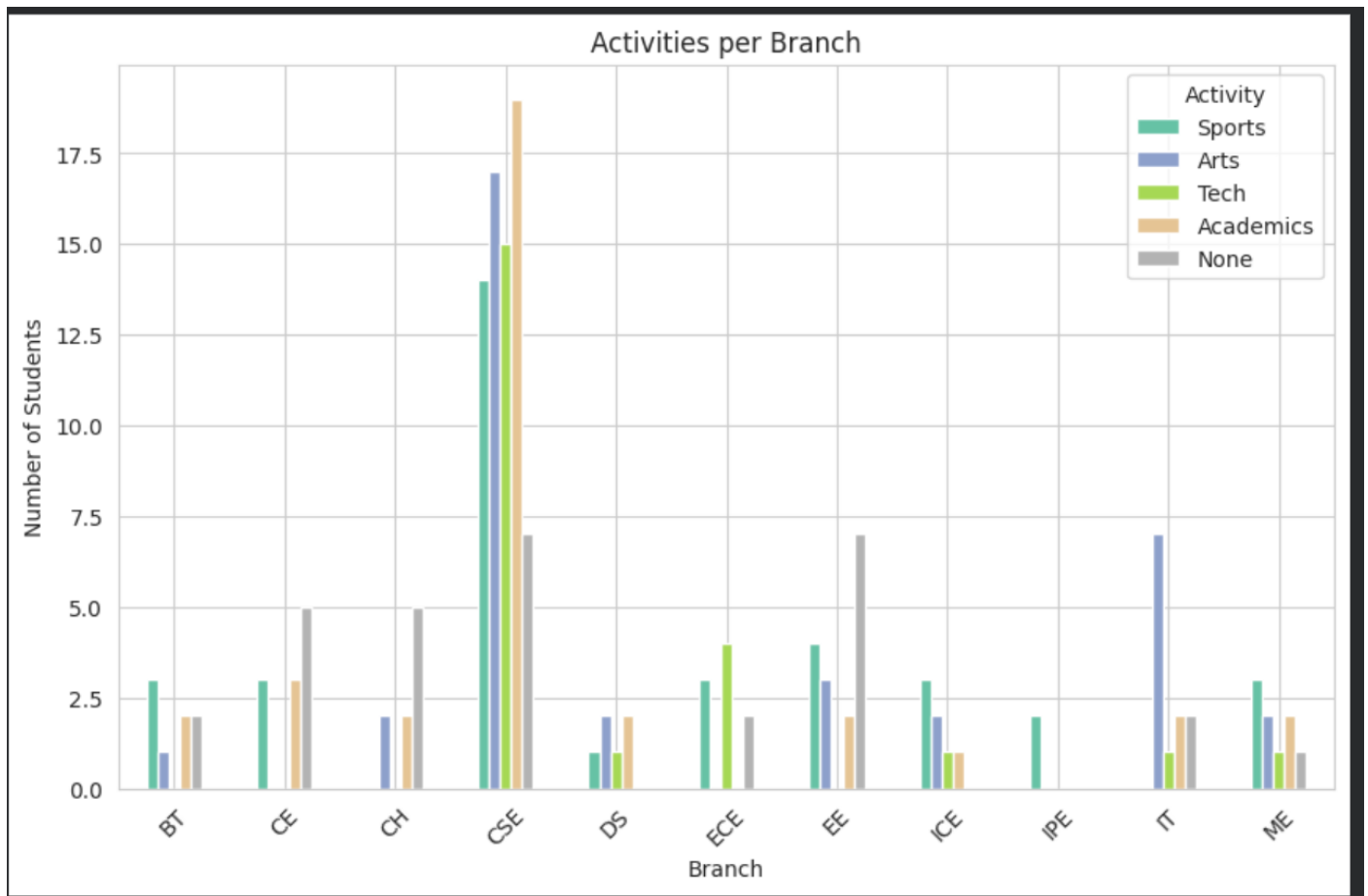
plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Between Activities')
plt.show()
```



Relating the activities with other columns

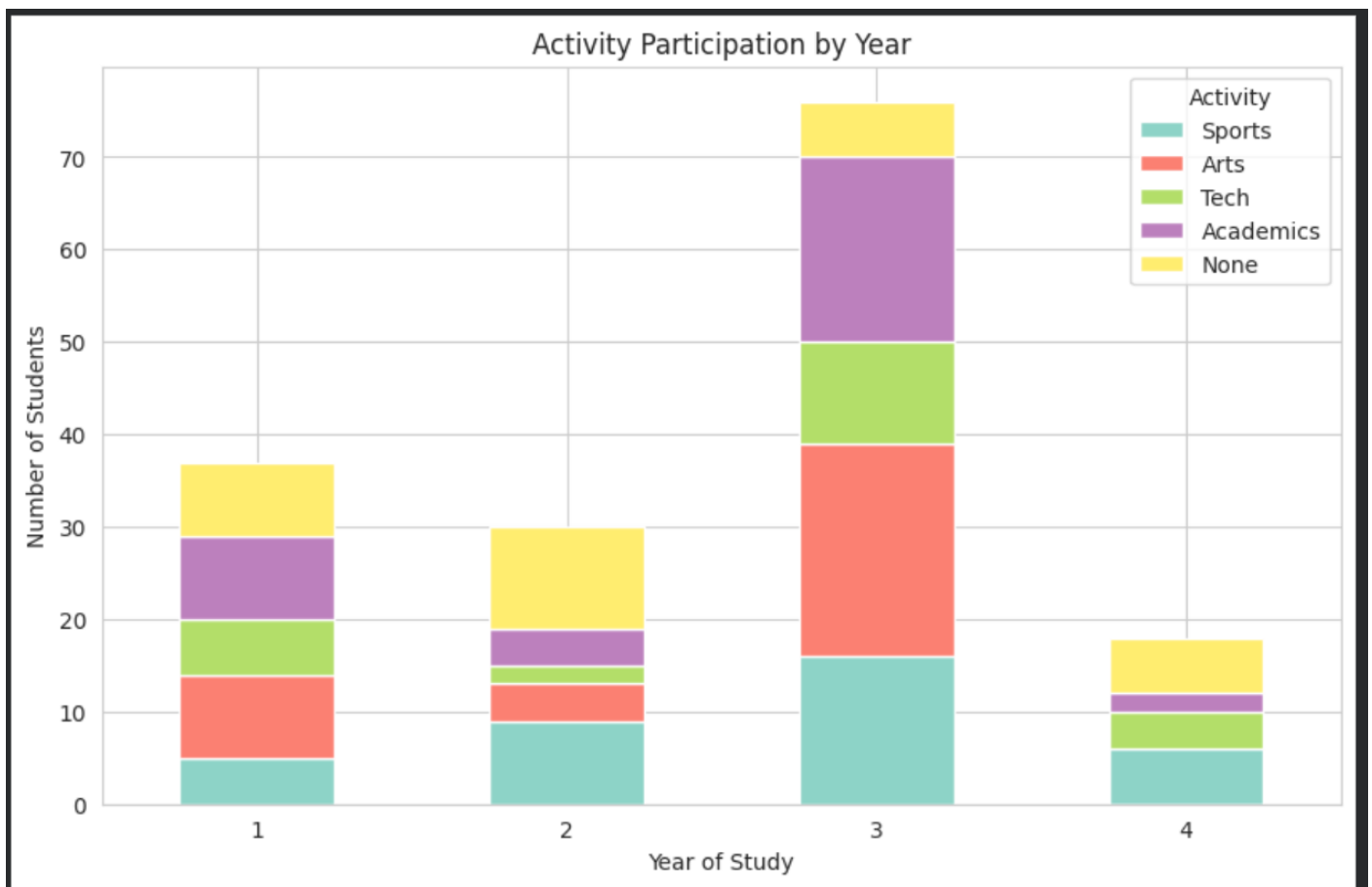
```
# Group by Branch and sum activities
branch_activity =
df.groupby('Branch')[['Sports','Arts','Tech','Academics','None']].sum().reset_index()

branch_activity.set_index('Branch').plot(kind='bar', figsize=(10,6), stacked=False,
colormap='Set2')
plt.title('Activities per Branch')
plt.ylabel('Number of Students')
plt.xlabel('Branch')
plt.xticks(rotation=45)
plt.legend(title='Activity')
plt
```



```
# Group by Year
year_activity = df.groupby('Year')[['Sports','Arts','Tech','Academics','None']].sum()

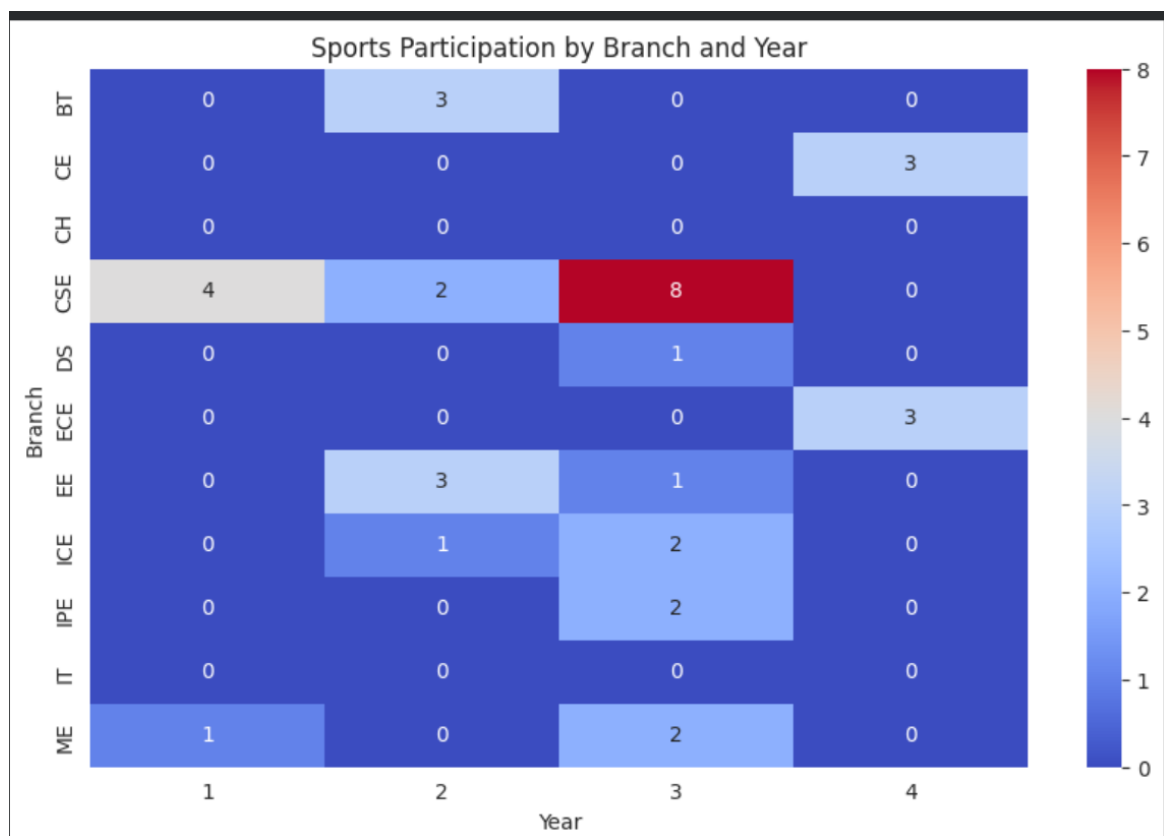
# Plot stacked bars
year_activity.plot(kind='bar', stacked=True, figsize=(10,6), colormap='Set3')
plt.title('Activity Participation by Year')
plt.ylabel('Number of Students')
plt.xlabel('Year of Study')
plt.xticks(rotation=0)
plt.legend(title='Activity')
plt.show()
```



```

pivot = df.pivot_table(index='Branch', columns='Year', values='Sports', aggfunc='sum',
fill_value=0)
sns.heatmap(pivot, annot=True, fmt='d', cmap='coolwarm')
plt.title('Sports Participation by Branch and Year')
plt.show()

```



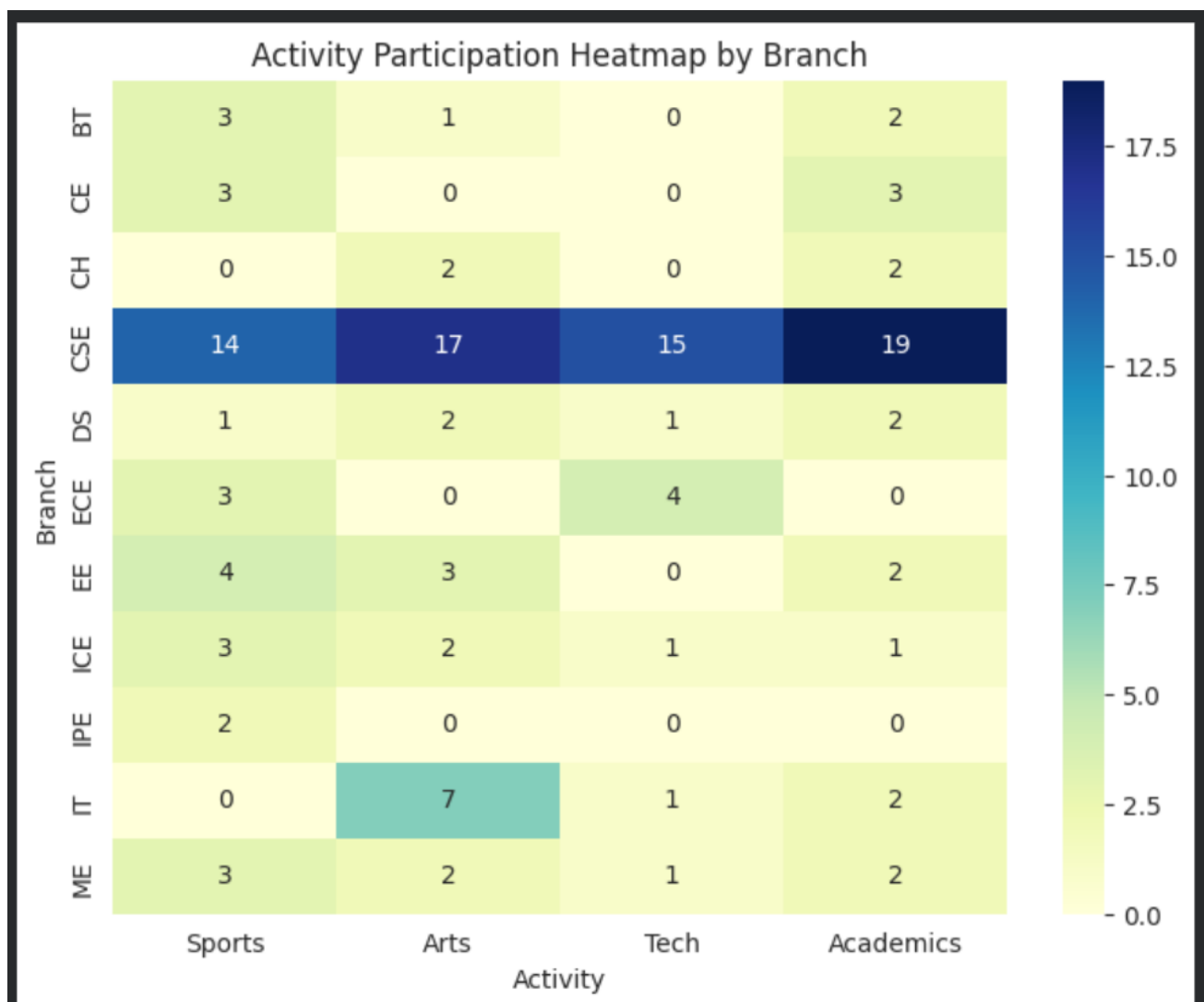
```

import seaborn as sns

# Correlation/participation per branch
branch_heat = df.groupby('Branch')[['Sports','Arts','Tech','Academics']].sum()

plt.figure(figsize=(8,6))
sns.heatmap(branch_heat, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Activity Participation Heatmap by Branch')
plt.xlabel('Activity')
plt.ylabel('Branch')
plt.show()

```



- **Stream_Choice**

```
# Clean column names
df.columns = df.columns.str.strip().str.replace('\n', '_')

# Cleaning function for Stream_Choice
def clean_stream(text):
    text = str(text).lower()
    text = unicode.unidecode(text)
    text = re.sub(r'[\^\w\s,()]/', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['Stream_Choice'] = df['Stream_Choice'].apply(clean_stream)

# Categorizing stream choices
stream_options = {
    'Own_Interest': ['own interest', 'interest', 'own'],
    'Parents_Suggestion': ['parents', 'relatives', 'sharma ji'],
    'Peer_Pressure': ['peer pressure', 'peer'],
    'No_Choice': ['no choice', 'fixed']
}

for col, keywords in stream_options.items():
    df[col] = df['Stream_Choice'].apply(lambda x: 1 if any(k in x for k in keywords) else 0)

df.drop(columns=['Stream_Choice'], inplace=True)
```

Explanation

The dataset contained an open-ended column named “**Stream_Choice**”, where students mentioned why they chose their current engineering stream (e.g., “own interest”, “parents told me”, “peer pressure”).

However, the text responses were inconsistent in wording and casing — for example, some wrote “OWN INTEREST”, others “my interest” or “parents suggested”.

To standardize this:

1. All text was converted to **lowercase** and cleaned using regex to remove unwanted symbols or punctuation.
2. We then defined key categories:
 - **Own_Interest:** Students who selected their stream by personal choice.
 - **Parents_Suggestion:** Streams influenced by parents or relatives.
 - **Peer_Pressure:** Choices made under friends’ or social pressure.
 - **No_Choice:** Students who had no option and were assigned by circumstances.

- Each of these was encoded into **binary columns** (1 for “Yes”, 0 for “No”), creating a clean, analyzable structure.
- The original text column was dropped for clarity.

This cleaning process allows us to **quantify student decision-making patterns** and later correlate them with **CGPA trends, motivation, and satisfaction**.

	Name	Year	Branch	School_Area	10th	12th	Current_CGPA	Previous_CGPA	CGPA_Change	CGPA_Reasons	...	12th_grade	Sports	Arts	Tech	A
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	7.2	7.5	Declined	Focus on other passions/projects	...	80-90%	0	0	1	
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	7.7	8	Declined	Time management issues	...	80-90%	0	1	0	
2	guri69	3	ICE	Semi-urban	Above 90%	Between 80% - 90%	7.7	7.5	Mix (up and down every semester)	Attendance rules	...	80-90%	1	1	0	
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	6.3	6.6	Declined	Friends / Peer influence	...	60-80%	0	0	0	
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	7.79	7.0(1st year)	Improved	Time management issues	...	>90%	1	0	0	
5	Ishmeen	1	CSE	Urban	Between 60% - 80%	Between 80% - 90%	9	1st	Improved	Interest in studies	...	80-90%	0	1	0	
6	Jayant	4	CE	Semi-urban	Between 60% - 80%	Between 80% - 90%	6.8	7.2	Declined	Focus on other passions/projects	...	80-90%	0	0	0	
7	Prakhar	2	BT	Rural	Under 60%	Under 60%	3.6	3.3	Improved	Health issues	...	<60%	1	0	0	
8	Ishaan88	3	IT	Rural	Under 60%	Between 60% - 80%	4.9	4.6	Improved	Lack of interest in branch	...	60-80%	0	1	0	
9	Hina99	2	EE	Urban	Between 60% - 80%	Between 80% - 90%	6.1	6.4	Declined	Time management issues	...	80-90%	1	0	0	
10	Saktimaan	3	ME	Rural	Between 60% - 80%	Between 60% - 80%	5.4	5.7	Declined	Lack of interest in branch	...	60-80%	0	0	0	
11	Hina	1	CH	Urban	Between 60% - 80%	Between 60% - 80%	5.5	5.2	Improved	Time management issues	...	60-80%	0	0	0	
12	Disha	1	CH	Rural	Between 60% - 80%	Between 60% - 80%	5.1	4.8	Improved	Time management issues	...	60-80%	0	0	0	
13	Mayank Don	3	DS	Urban	Above 90%	Above 90%	7.36	6.8	Improved	Interest in studies	...	>90%	1	1	0	

Cleaning CGPA Reasons and Categorization

```
# Clean column names
df.columns = df.columns.str.strip().str.replace('\n', '_')

def clean_reason(text):
    text = str(text).lower()
    text = unicode.unidecode(text)
    text = re.sub(r'[^\w\s,()]\s', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['CGPA_Reasons'] = df['CGPA_Reasons'].apply(clean_reason)

cgpa_reasons_options = {
    'Interest_in_studies': ['interest in studies', 'interest'],
    'Time_management': ['time management', 'time management issues'],
    'Attendance': ['attendance rules', 'attendance'],
    'Peer_influence': ['friends', 'peer influence', 'peer'],
    'Health_issues': ['health issues', 'health'],
    'Lack_of_interest_in_branch': ['lack of interest in branch', 'lack of interest'],
    'Other_passions': ['focus on other passions', 'projects', 'other passions']
}

for col, keywords in cgpa_reasons_options.items():
    df[col] = df['CGPA_Reasons'].apply(lambda x: 1 if any(k in x for k in keywords) else 0)

df.drop(columns=['CGPA_Reasons'], inplace=True)

df.head(20)
```

	Name	Year	Branch	School_Area	10th	12th	Current_CGPA	Previous_CGPA	CGPA_Change	Rate our College	...	Parents_Suggestion	Peer_Pressure
0	Mahi	4	ECE	Urban	Between 60% - 80%	Between 80% - 90%	7.2	7.5	Declined	4	...	0	0
1	Bhav Na Kha	3	IT	Semi-urban	Between 80% - 90%	Between 80% - 90%	7.7	8	Declined	3	...	0	0
2	guri69	3	ICE	Semi-urban	Above 90%	Between 80% - 90%	7.7	7.5	Mix (up and down every semester)	3	...	0	0
3	Falguni2002	4	CE	Urban	Between 60% - 80%	Between 60% - 80%	6.3	6.6	Declined	2	...	0	0
4	Plaksha Kapoor	3	IPE	Semi-urban	Above 90%	Above 90%	7.79	7.0(1st year)	Improved	2	...	0	0
5	Ishmeen	1	CSE	Urban	Between 60% - 80%	Between 80% - 90%	9	1st	Improved	4	...	0	0
6	Jayant	4	CE	Semi-urban	Between 60% - 80%	Between 80% - 90%	6.8	7.2	Declined	3	...	0	0

Explanation

Similarly, the “**CGPA_Reasons**” column collected student feedback about what factors influenced their academic performance.

Students mentioned multiple causes such as “*health issues*”, “*peer distraction*”, “*focus on projects*”, or “*attendance problems*”.

To structure this information:

- The text was cleaned using `clean_reason()` to normalize casing and remove noise.
- We defined **7 main influencing categories** — Interest in studies, Time management, Attendance, Peer influence, Health issues, Lack of interest in branch, and Focus on other passions/projects.
- Each response was analyzed for keywords and categorized under these columns.
- The result was a **binary dataset** clearly showing the proportion of students affected by each factor.

This transformation helps in **identifying root causes of low or fluctuating CGPA**, and later correlating them with other attributes like **stream choice** and **motivation levels**.

Visualize it:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10,6)

# 1 Pie Charts for School Activities
activity_cols = ['Sports', 'Arts', 'Tech', 'Academics', 'None']
activity_counts = df[activity_cols].sum()

plt.figure(figsize=(8,8))
plt.pie(activity_counts.values, labels=activity_counts.index, autopct='%1.1f%%',
        colors=sns.color_palette('Set2'))
plt.title('Distribution of School Activities')
plt.show()

# 2 Pie Chart for Stream Choice
stream_cols = ['Own_Interest', 'Parents_Suggestion', 'Peer_Pressure', 'No_Choice']
stream_counts = df[stream_cols].sum()

plt.figure(figsize=(8,8))
plt.pie(stream_counts.values, labels=stream_counts.index, autopct='%1.1f%%',
        colors=sns.color_palette('Set1'))
plt.title('Distribution of Stream Choice')
plt.show()

# 3 Bar Charts: Activities by Branch
branch_activity = df.groupby('Branch')[activity_cols].sum()
branch_activity.plot(kind='bar', stacked=False, colormap='Set2', figsize=(12,6))
plt.title('School Activities by Branch')
```

```
plt.ylabel('Number of Students')
plt.xlabel('Branch')
plt.xticks(rotation=45)
plt.show()
```

4 Bar Charts: Stream Choice by Branch

```
branch_stream = df.groupby('Branch')[stream_cols].sum()
branch_stream.plot(kind='bar', stacked=True, colormap='Set1', figsize=(12,6))
plt.title('Stream Choice by Branch')
plt.ylabel('Number of Students')
plt.xlabel('Branch')
plt.xticks(rotation=45)
plt.show()
```

5 CGPA Distribution by Activities

```
df_melt_activity = df.melt(id_vars=['Current_CGPA'], value_vars=activity_cols,
                           var_name='Activity', value_name='Participated')
df_melt_activity = df_melt_activity[df_melt_activity['Participated']==1]

plt.figure(figsize=(10,6))
sns.boxplot(x='Activity', y='Current_CGPA', data=df_melt_activity, palette='Set2')
plt.title('CGPA Distribution by School Activities')
plt.show()
```

6 CGPA Distribution by Stream Choice

```
df_melt_stream = df.melt(id_vars=['Current_CGPA'], value_vars=stream_cols,
                        var_name='StreamChoice', value_name='Selected')
df_melt_stream = df_melt_stream[df_melt_stream['Selected']==1]

plt.figure(figsize=(10,6))
sns.boxplot(x='StreamChoice', y='Current_CGPA', data=df_melt_stream, palette='Set1')
plt.title('CGPA Distribution by Stream Choice')
plt.show()
```

7 Academic Scores vs Stream Choice (Scatter + Box)

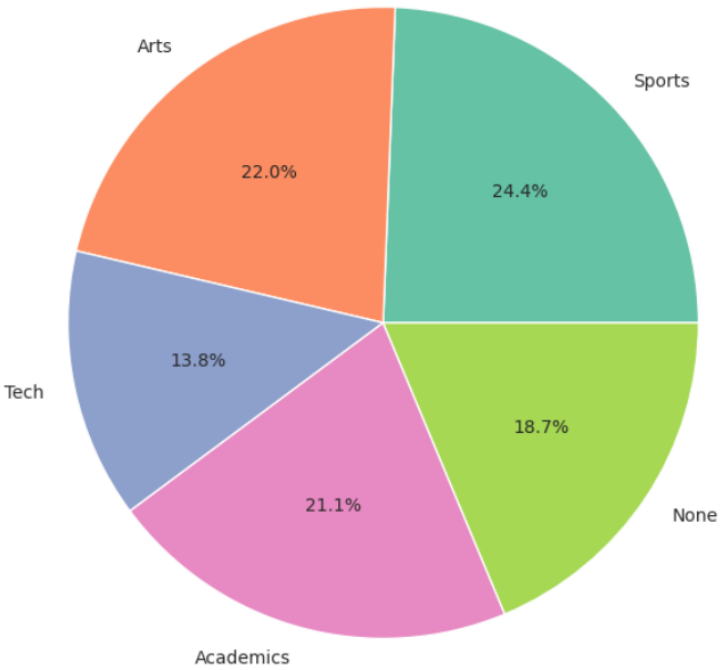
```
plt.figure(figsize=(12,6))
sns.scatterplot(x='12th', y='Current_CGPA', hue='Own_Interest', data=df, palette={1:'green',0:'red'})
plt.title('12th Percentage vs CGPA (Own Interest)')
plt.xlabel('12th Percentage / CGPA Equivalent')
plt.ylabel('Current CGPA')
plt.show()
```

```
plt.figure(figsize=(12,6))
sns.boxplot(x='Own_Interest', y='Current_CGPA', data=df, palette='Set2')
plt.title('CGPA Distribution by Own Interest Stream Choice')
plt.xticks([0,1], ['No', 'Yes'])
plt.show()
```

8 Heatmap: Correlation of Academic Scores

```
numeric_cols = ['10th','12th','Current_CGPA','Previous_CGPA','CGPA_Change']
plt.figure(figsize=(8,6))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Between Academic Scores')
plt.show()
```

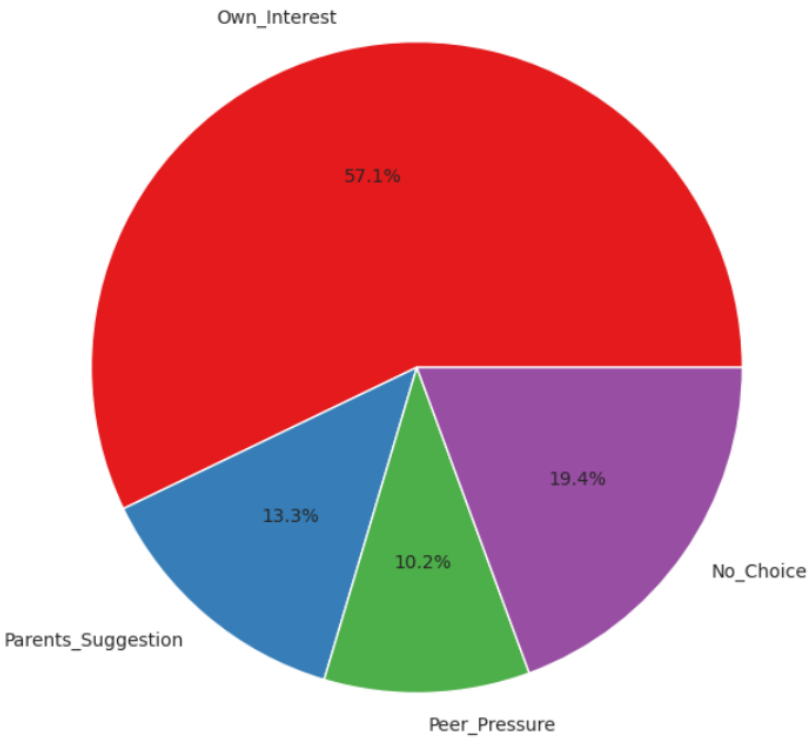
Distribution of School Activities



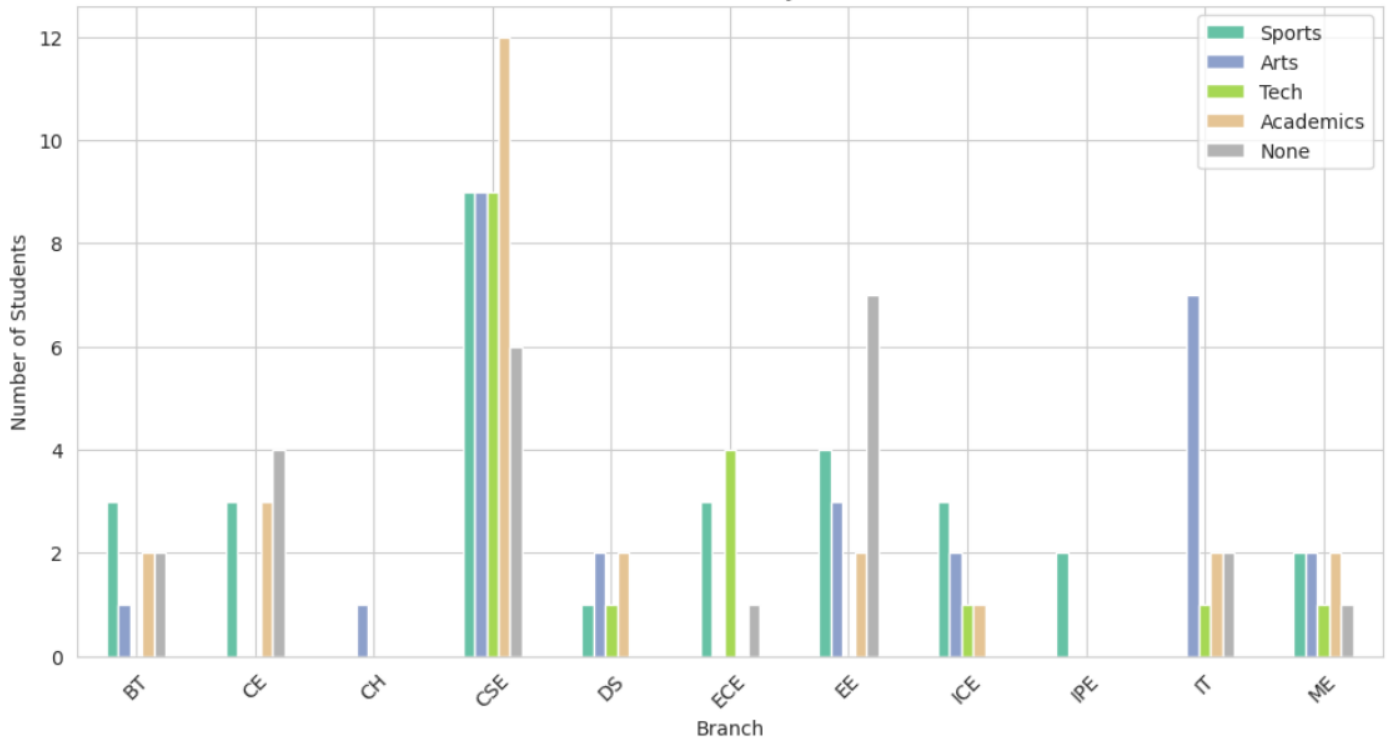
Distribution of Stream Choice

.

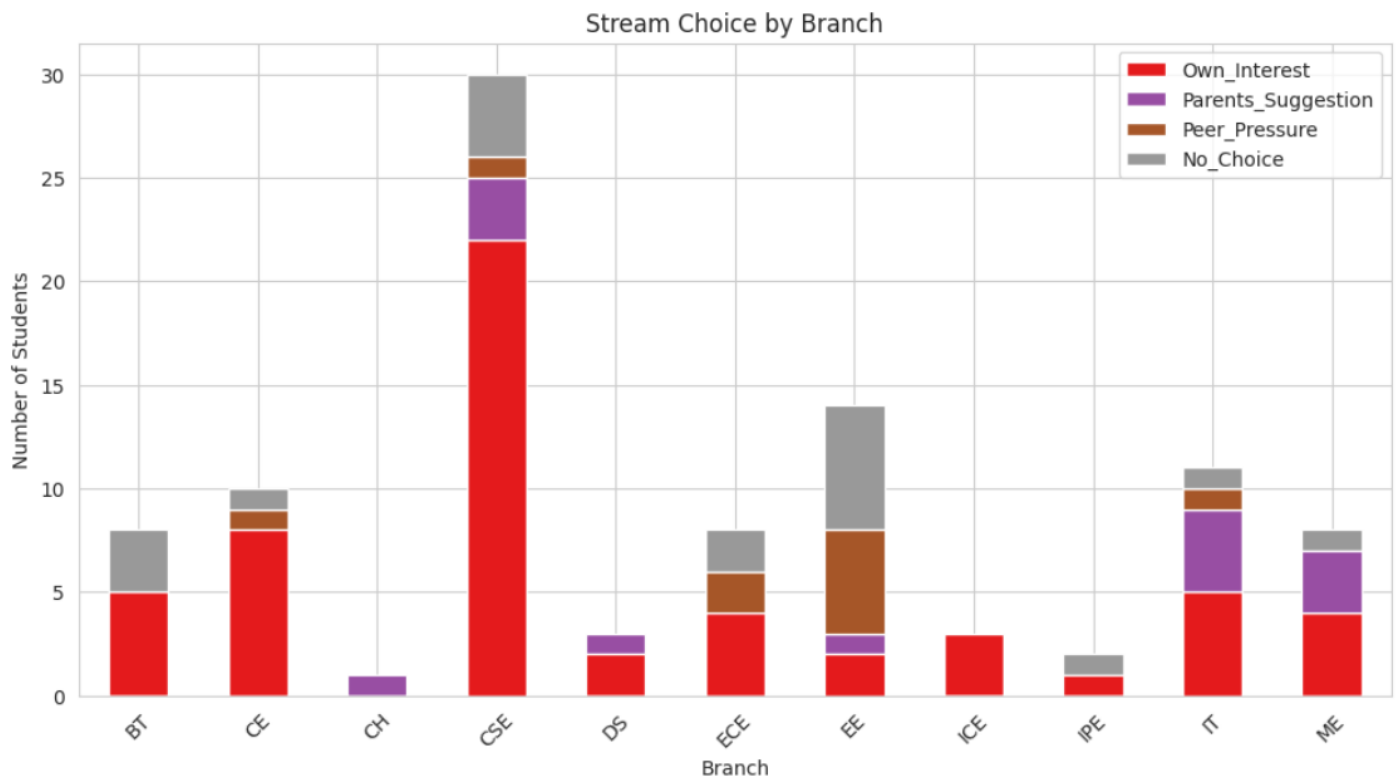
Distribution of Stream Choice



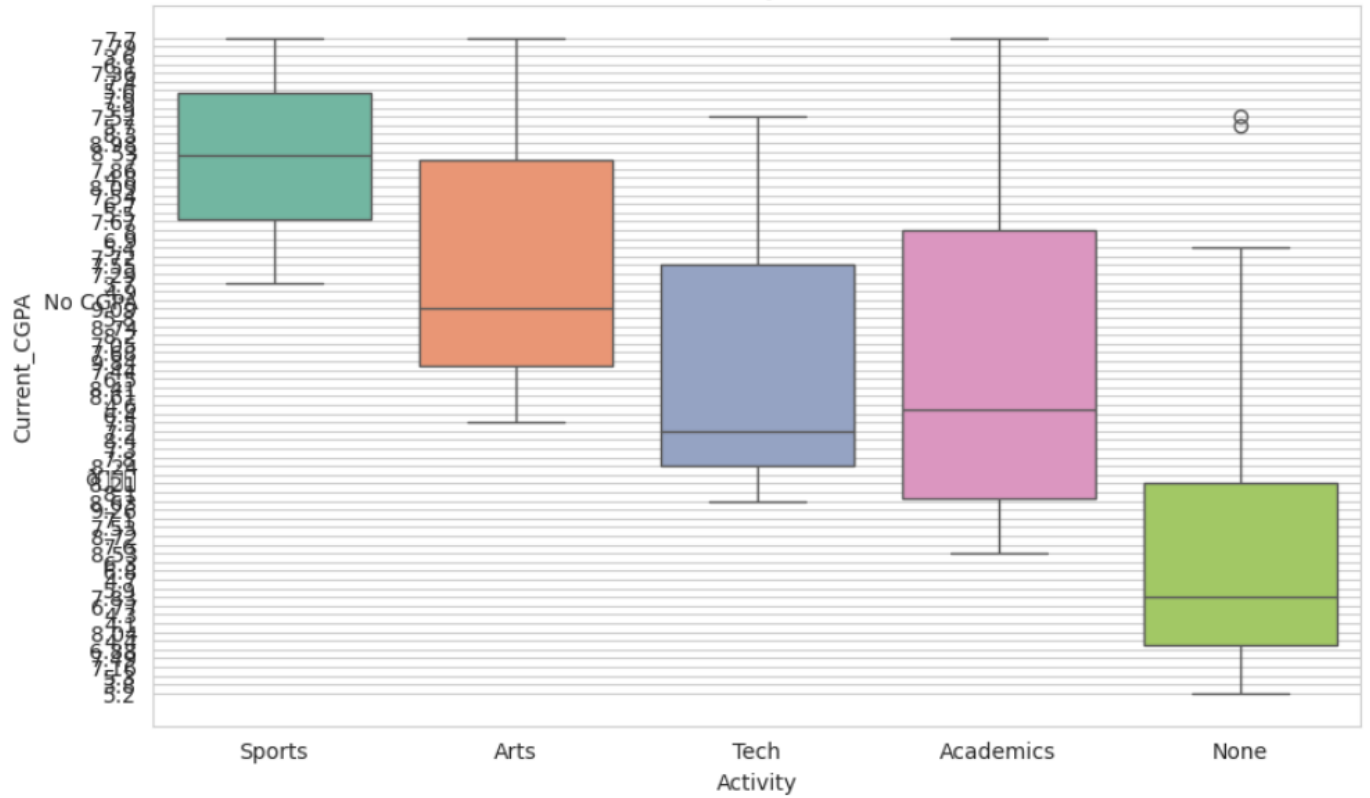
School Activities by Branch



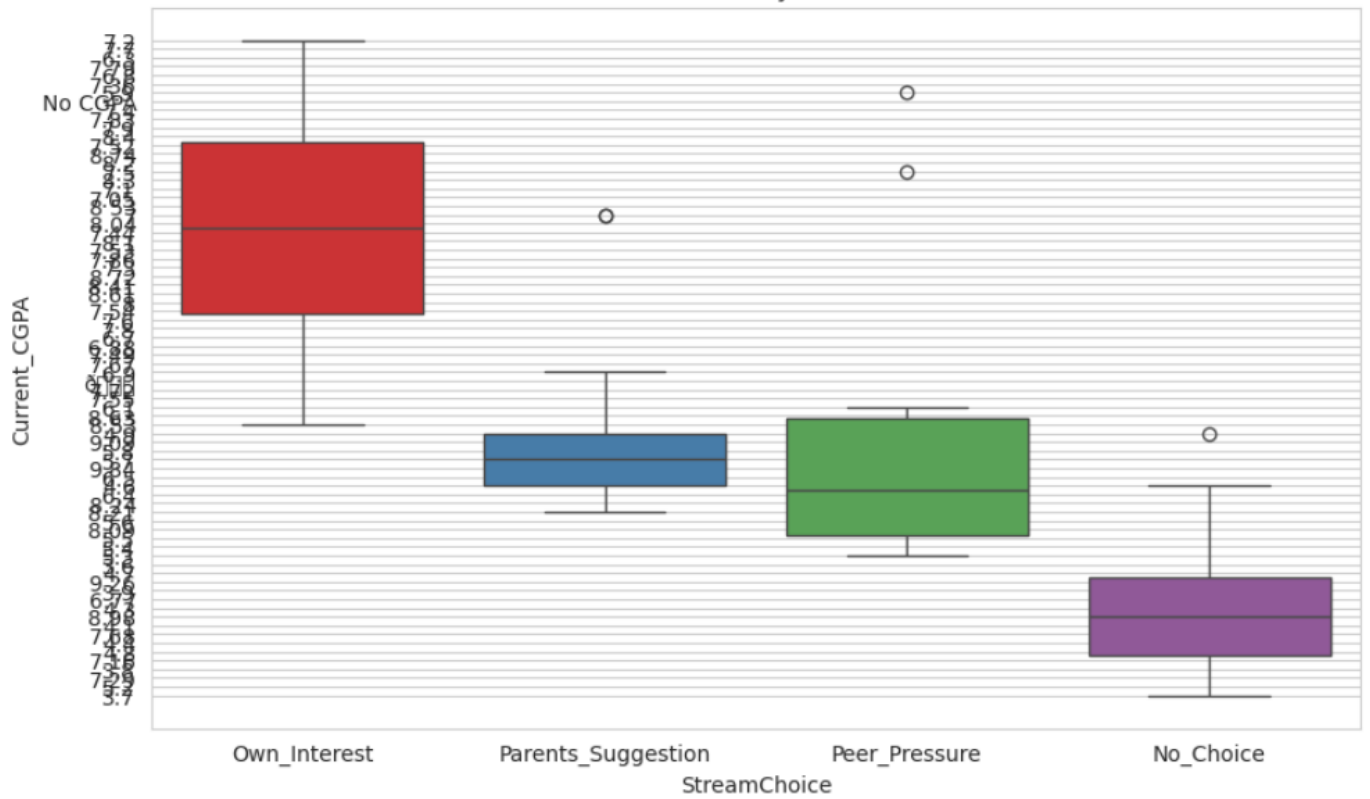
Stream Choice by Branch

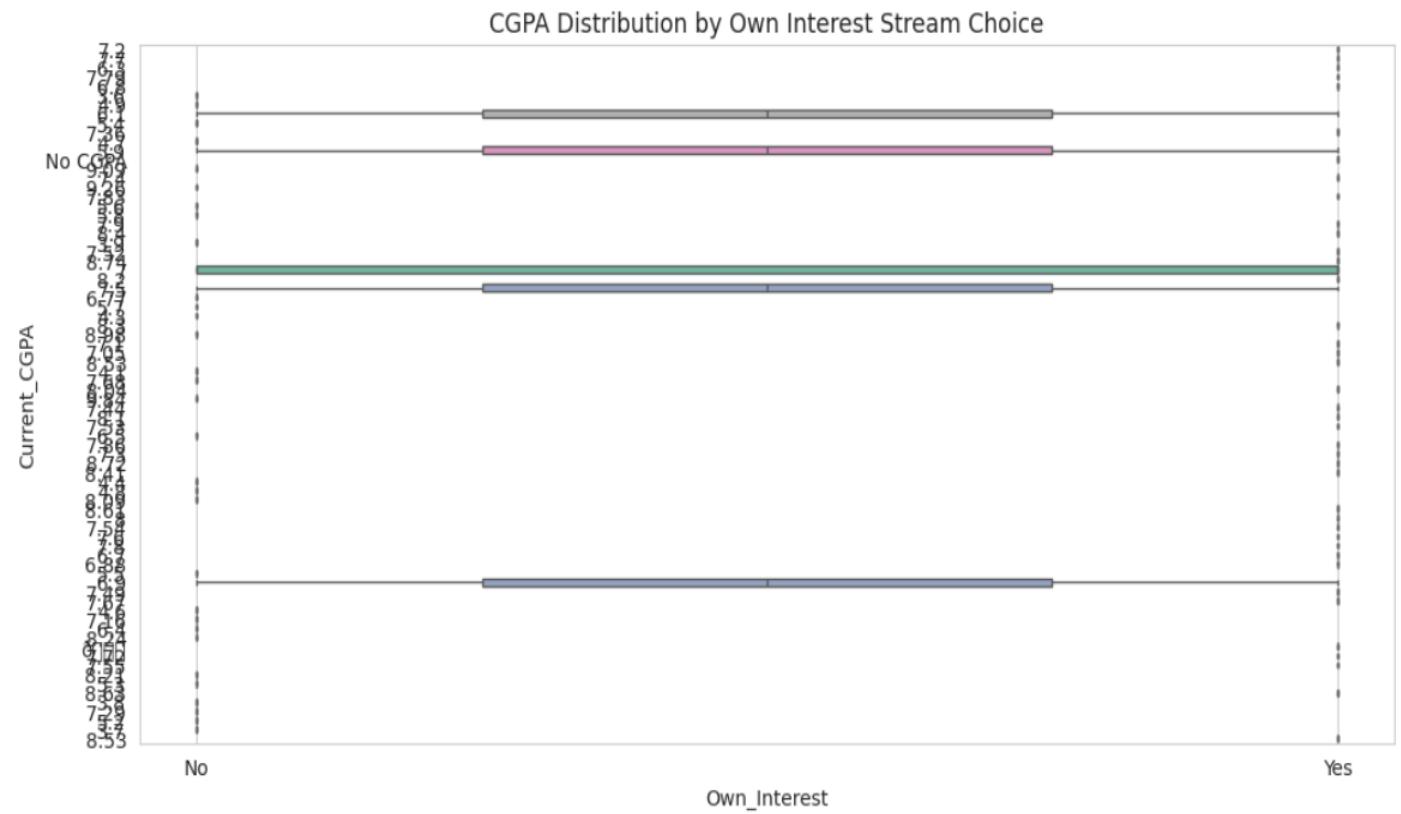
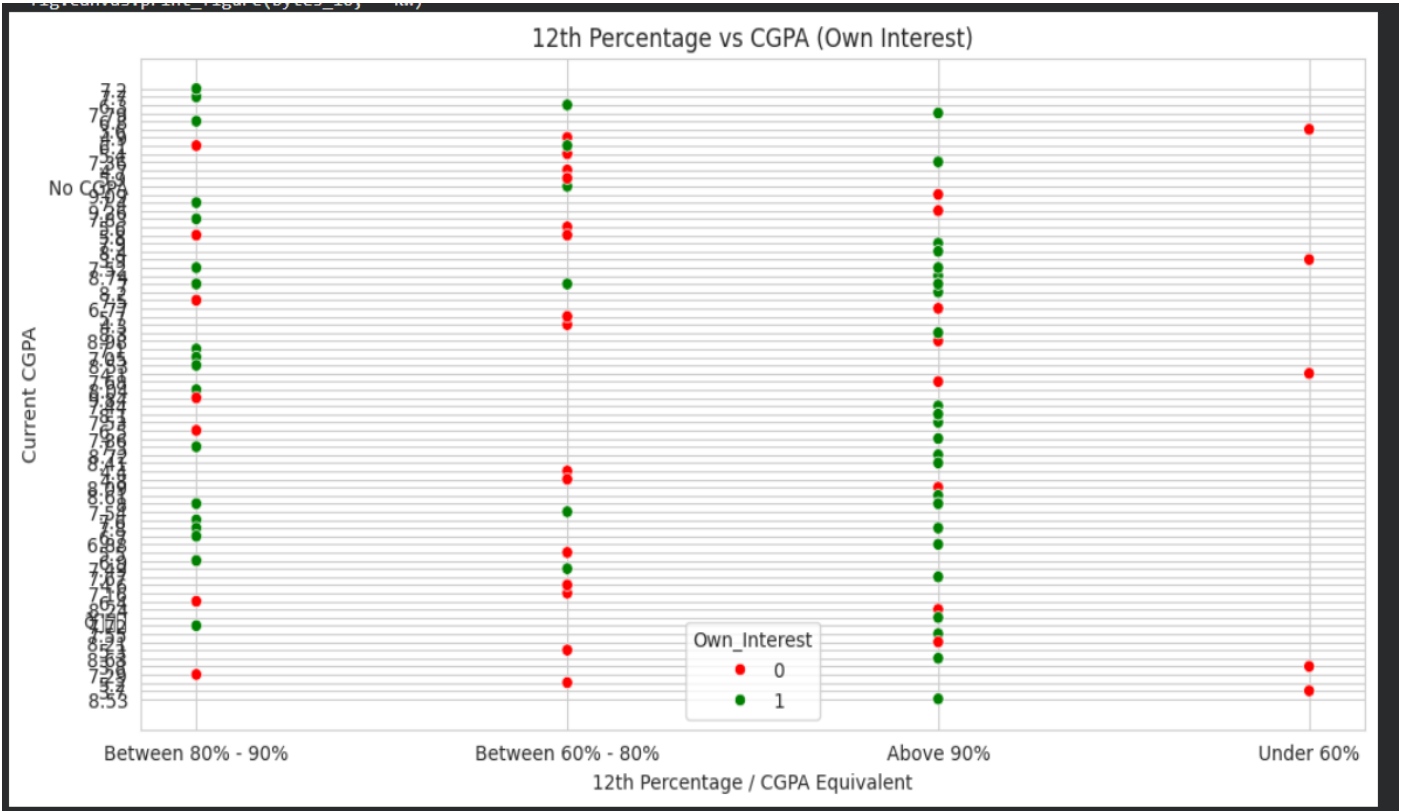


CGPA Distribution by School Activities



CGPA Distribution by Stream Choice





- **Current_CGPA**

```
# Convert Current_CGPA to numeric
df['Current_CGPA'] = pd.to_numeric(df['Current_CGPA'], errors='coerce')

df = df.dropna(subset=['Current_CGPA']).reset_index(drop=True)

bins = [0, 6, 7, 8, 9, 10]
labels = ['<6', '6-7', '7-8', '8-9', '>9']
df['CGPA_Range'] = pd.cut(df['Current_CGPA'], bins=bins, labels=labels, include_lowest=True)

df[['Current_CGPA', 'CGPA_Range']].head(10)
```

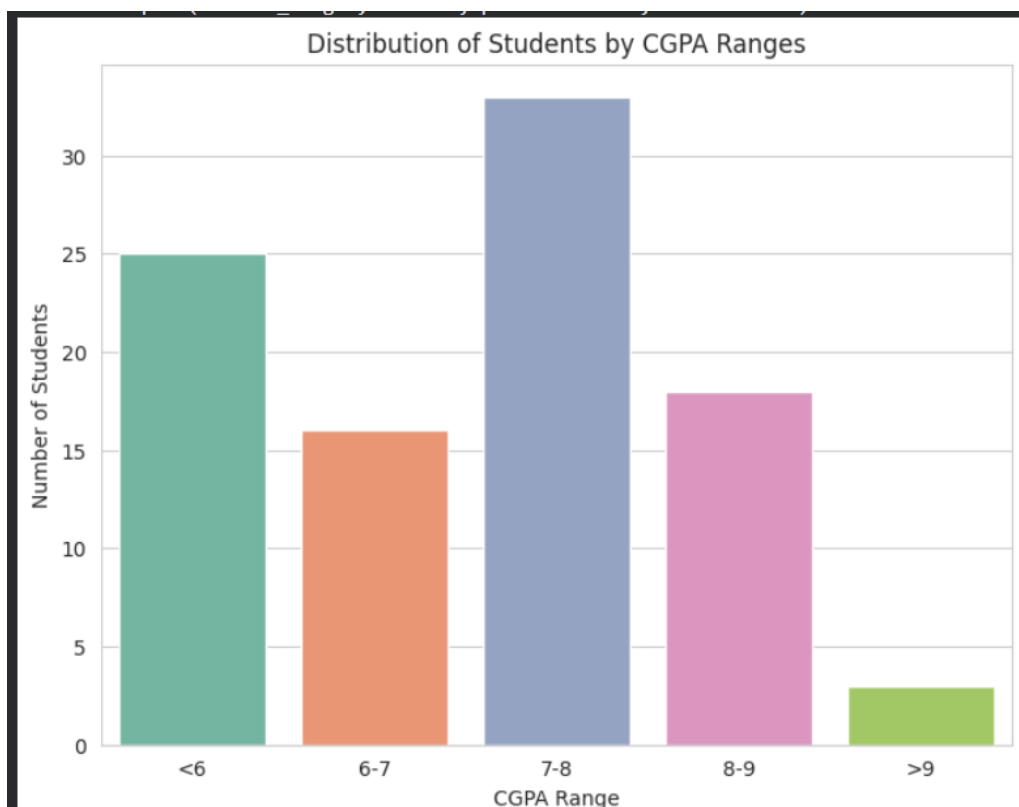
Visualize it:

```
# Convert Current_CGPA to numeric
df['Current_CGPA'] = pd.to_numeric(df['Current_CGPA'], errors='coerce')

df = df.dropna(subset=['Current_CGPA']).reset_index(drop=True)

bins = [0, 6, 7, 8, 9, 10]
labels = ['<6', '6-7', '7-8', '8-9', '>9']
df['CGPA_Range'] = pd.cut(df['Current_CGPA'], bins=bins, labels=labels, include_lowest=True)

df[['Current_CGPA', 'CGPA_Range']].head(10)
```



```
# CGPA Range vs Activities
```

```
activity_cols = ['Sports','Arts','Tech','Academics','None']
```

```
for act in activity_cols:
```

```
    plt.figure(figsize=(8,6))
```

```
    sns.countplot(x='CGPA_Range', hue=act, data=df, palette='Set2', order=labels)
```

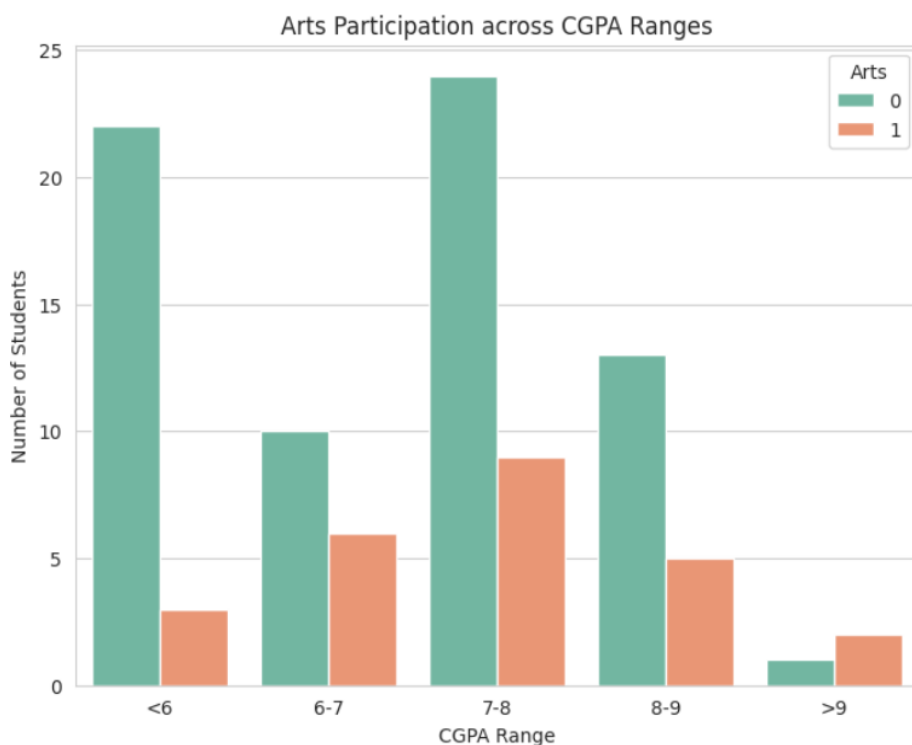
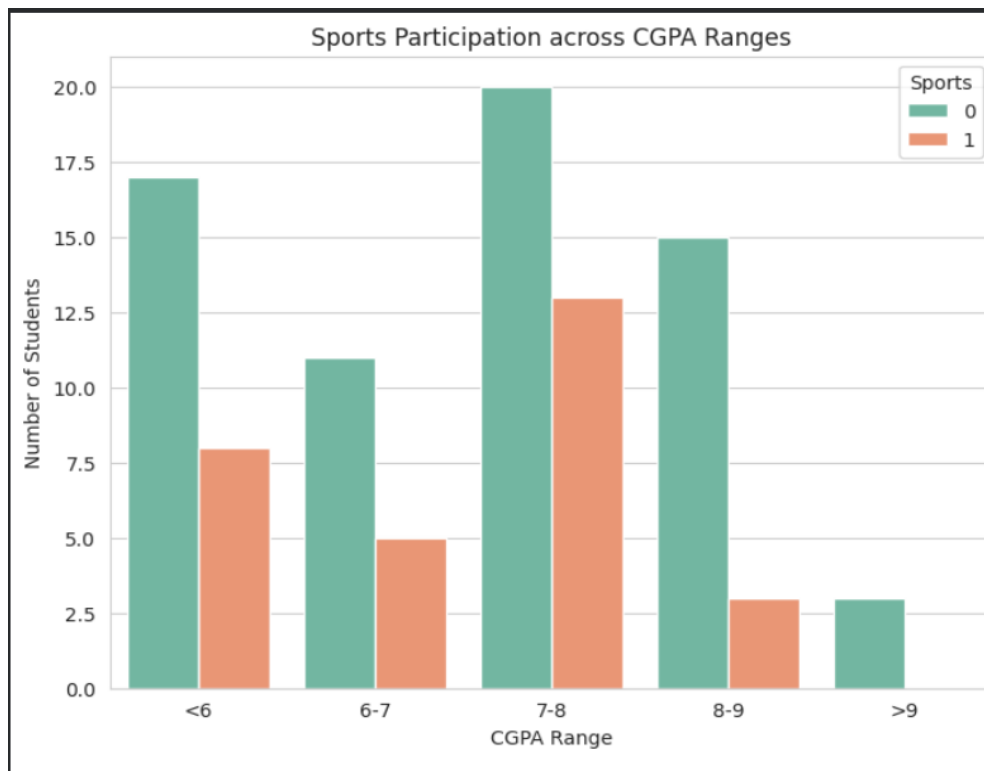
```
    plt.title(f'{act} Participation across CGPA Ranges')
```

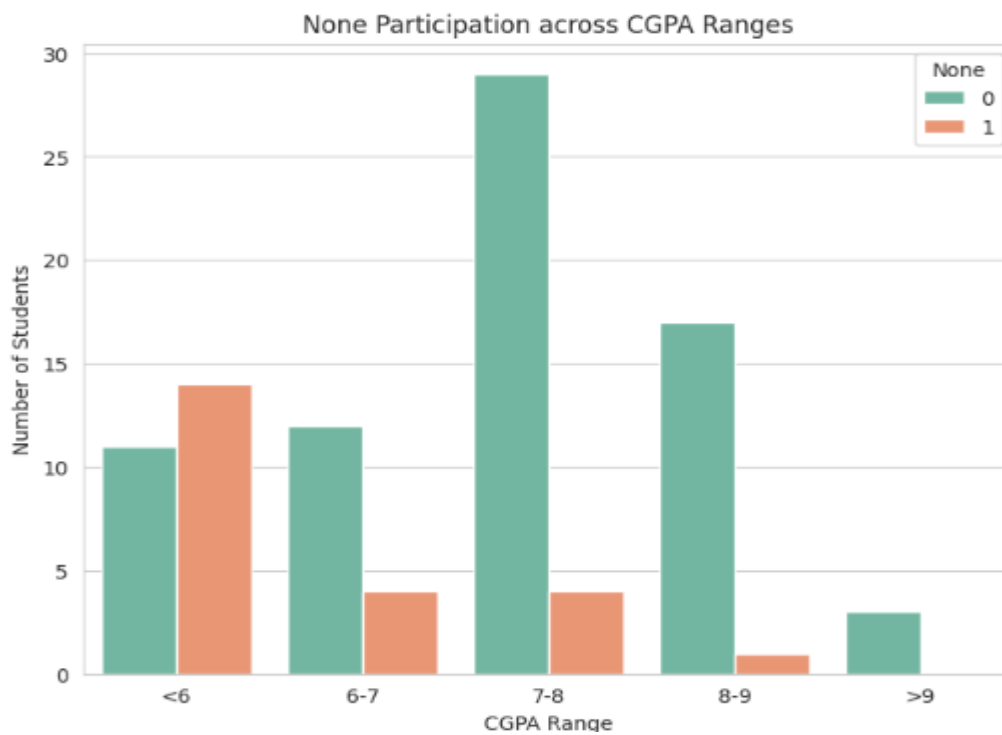
```
    plt.ylabel('Number of Students')
```

```
    plt.xlabel('CGPA Range')
```

```
    plt.legend(title=act)
```

```
    plt.show()
```





#CGPA Range vs Stream Choice

```
stream_cols = ['Own_Interest', 'Parents_Suggestion', 'Peer_Pressure', 'No_Choice']
```

```
for sc in stream_cols:
```

```
    plt.figure(figsize=(8,6))
```

```
    sns.countplot(x='CGPA_Range', hue=sc, data=df, palette='Set1', order=labels)
```

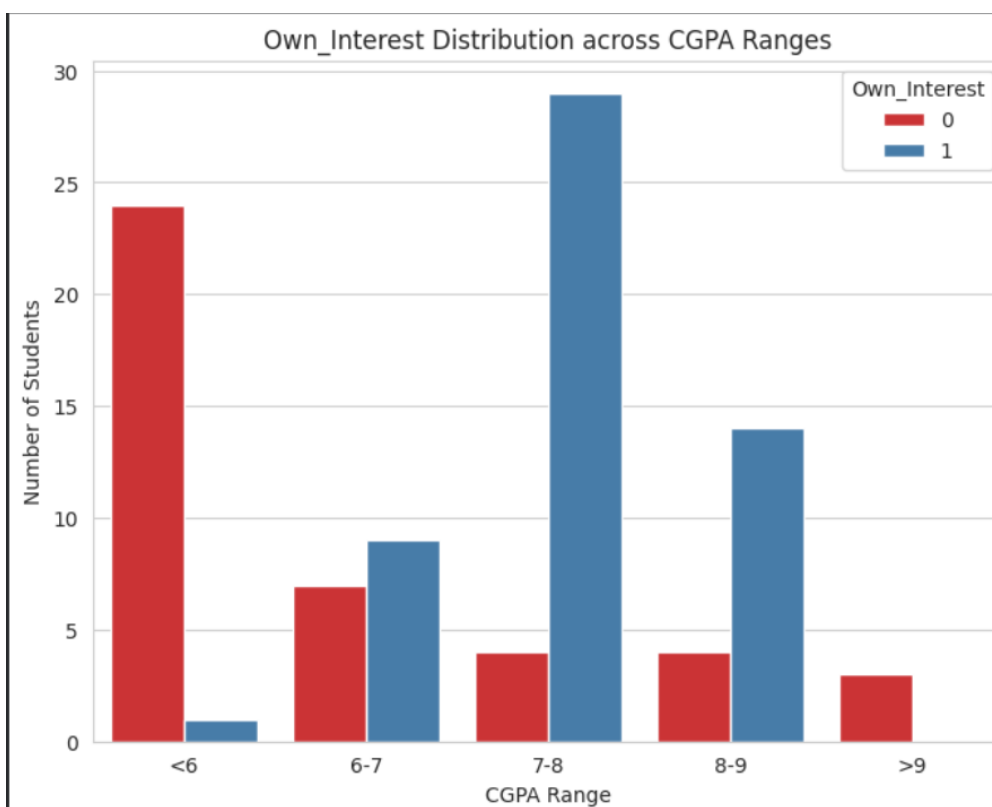
```
    plt.title(f'{sc} Distribution across CGPA Ranges')
```

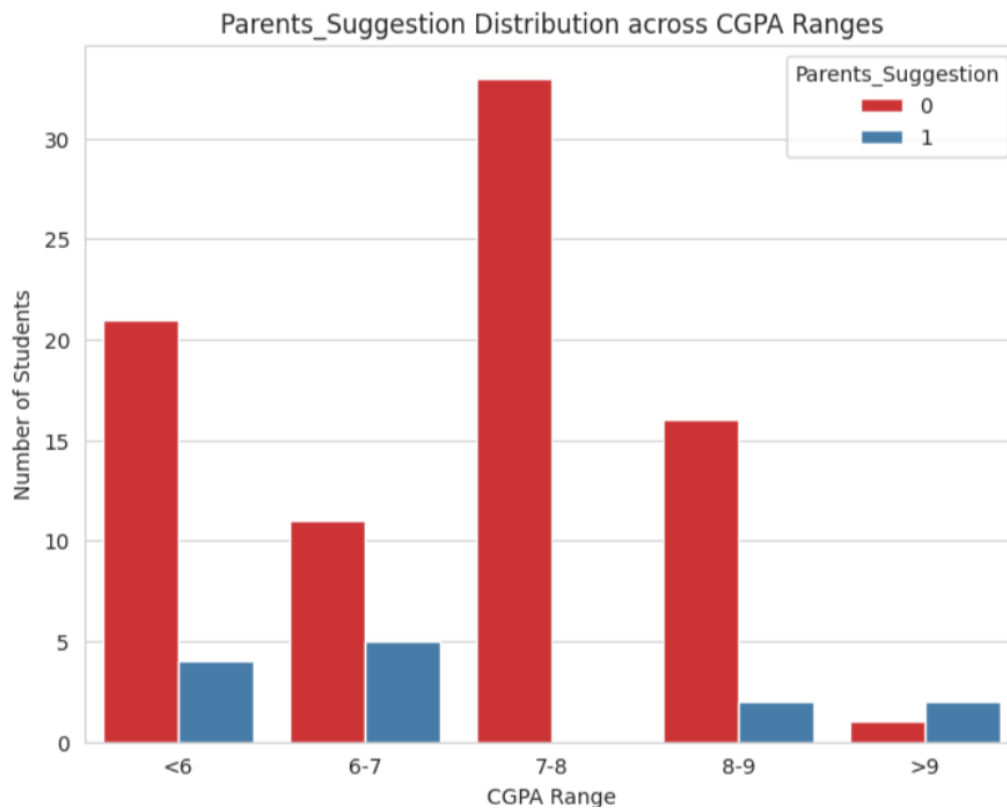
```
    plt.ylabel('Number of Students')
```

```
    plt.xlabel('CGPA Range')
```

```
    plt.legend(title=sc)
```

```
    plt.show()
```





- **CGPA Change**

```
import pandas as pd
import re
import unicode

def clean_change(text):
    text = str(text).lower()
    text = unicode.unidecode(text)
    text = re.sub(r'^[a-z\s]', '', text)
    text = text.strip()

    if 'mix' in text:
        return 'Mix'
    elif 'improved' in text:
        return 'Improved'
    elif 'declined' in text:
        return 'Declined'
    else:
        return 'Other'

df['CGPA_Change'] = df['CGPA_Change'].apply(clean_change)
df.head(10)
```

10th	12th	Current_CGPA	Previous_CGPA	CGPA_Change	C
Between 60% - 80%	Between 80% - 90%	7.20	7.5	Declined	
Between 80% - 90%	Between 80% - 90%	7.70	8	Declined	
Above 90%	Between 80% - 90%	7.70	7.5	Mix	
Between 60% - 80%	Between 60% - 80%	6.30	6.6	Declined	
Above 90%	Above 90%	7.79	7.0(1st year)	Improved	
Between 60% - 80%	Between 80% - 90%	6.80	7.2	Declined	
Under 60%	Under 60%	3.60	3.3	Improved	
Under 60%	Between 60% - 80%	4.90	4.6	Improved	
Between 60% - 80%	Between 80% - 90%	6.10	6.4	Declined	

```

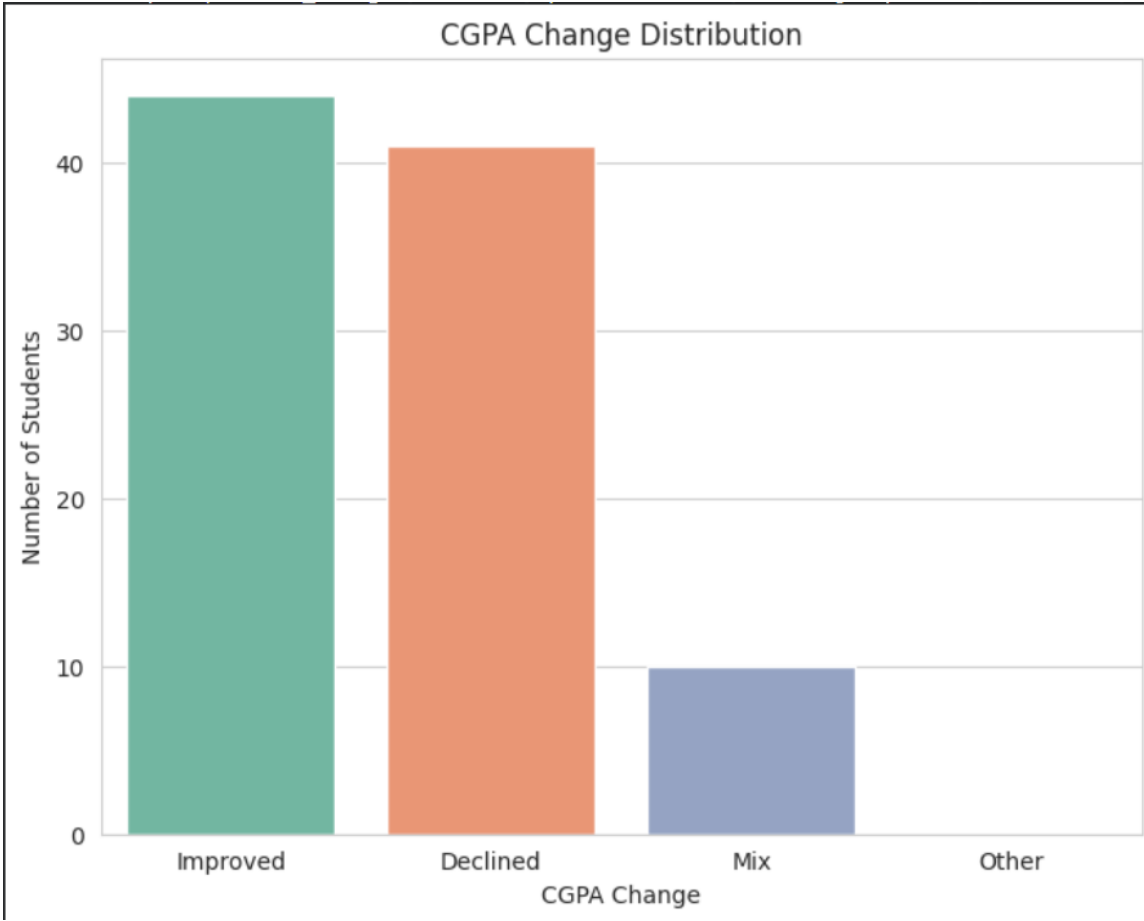
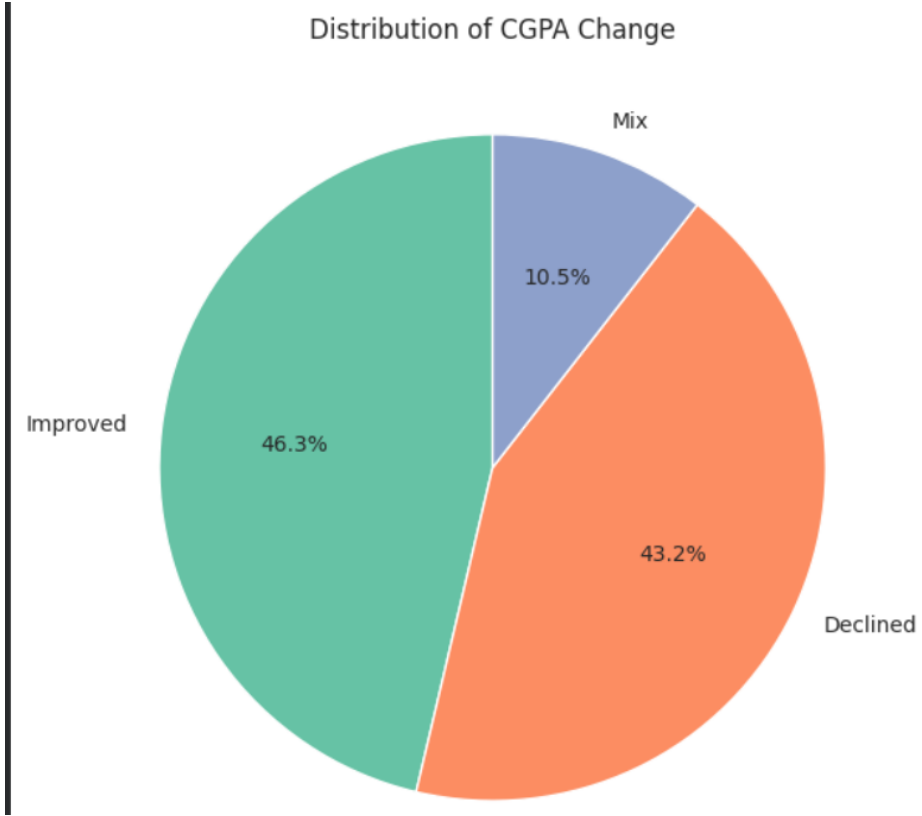
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

# Pie Chart
plt.figure(figsize=(7,7))
df['CGPA_Change'].value_counts().plot.pie(autopct='%1.1f%%',
colors=sns.color_palette('Set2'), startangle=90)
plt.title('Distribution of CGPA Change')
plt.ylabel("")
plt.show()

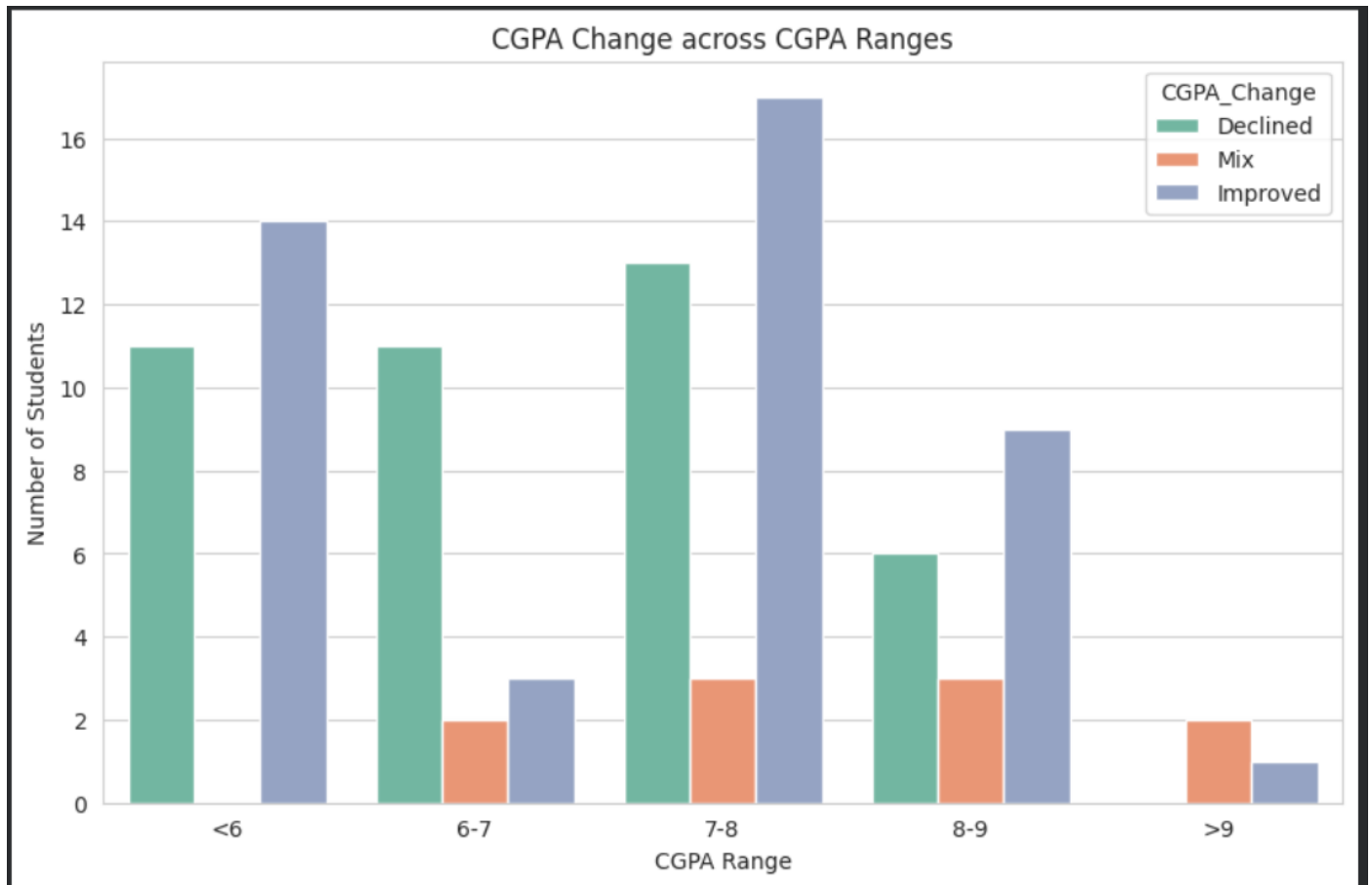
# Bar Chart
plt.figure(figsize=(8,6))
sns.countplot(x='CGPA_Change', data=df, palette='Set2',
order=['Improved','Declined','Mix','Other'])
plt.title('CGPA Change Distribution')
plt.ylabel('Number of Students')
plt.xlabel('CGPA Change')
plt.show()

```



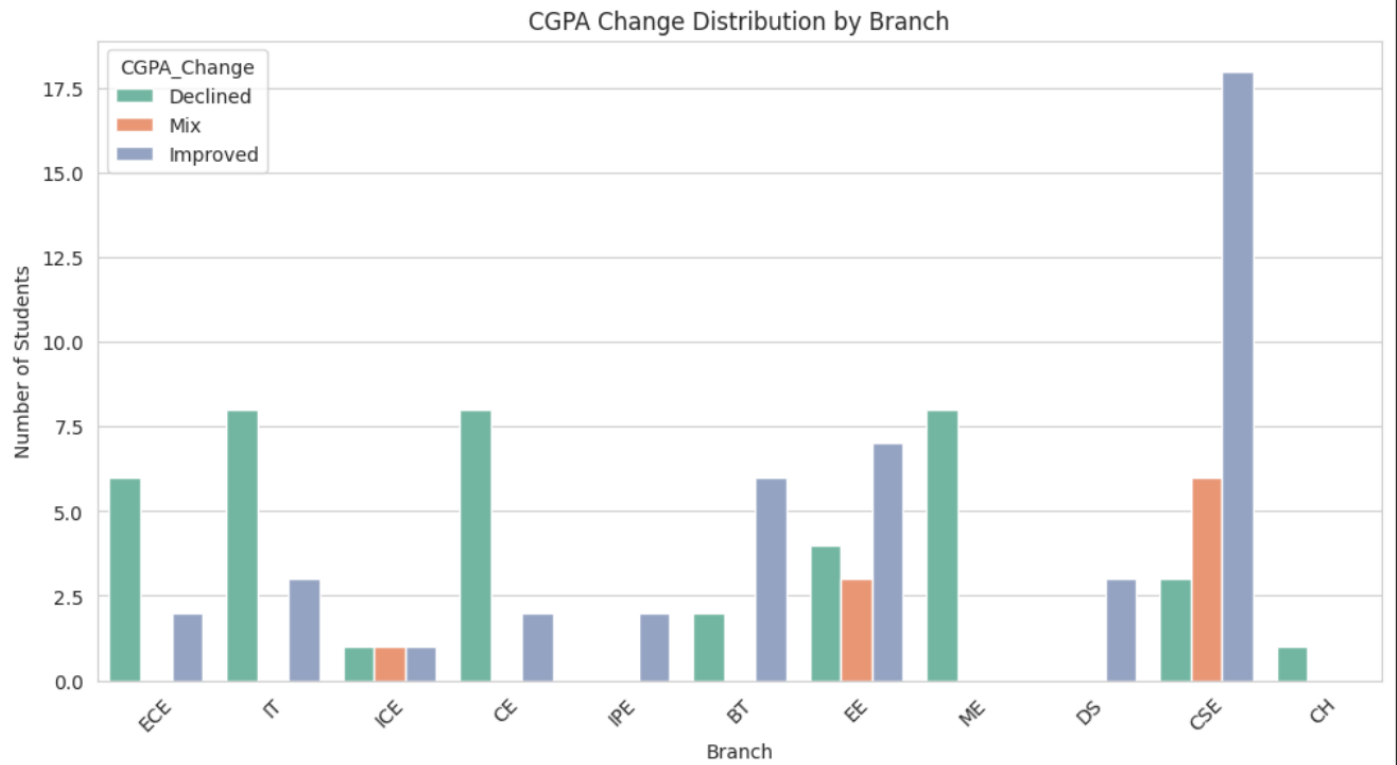
```
# CGPA_Change vs CGPA_Range
```

```
plt.figure(figsize=(10,6))
sns.countplot(x='CGPA_Range', hue='CGPA_Change', data=df, palette='Set2', order=['<6','6-7','7-8','8-9','>9'])
plt.title('CGPA Change across CGPA Ranges')
plt.ylabel('Number of Students')
plt.xlabel('CGPA Range')
plt.show()
```



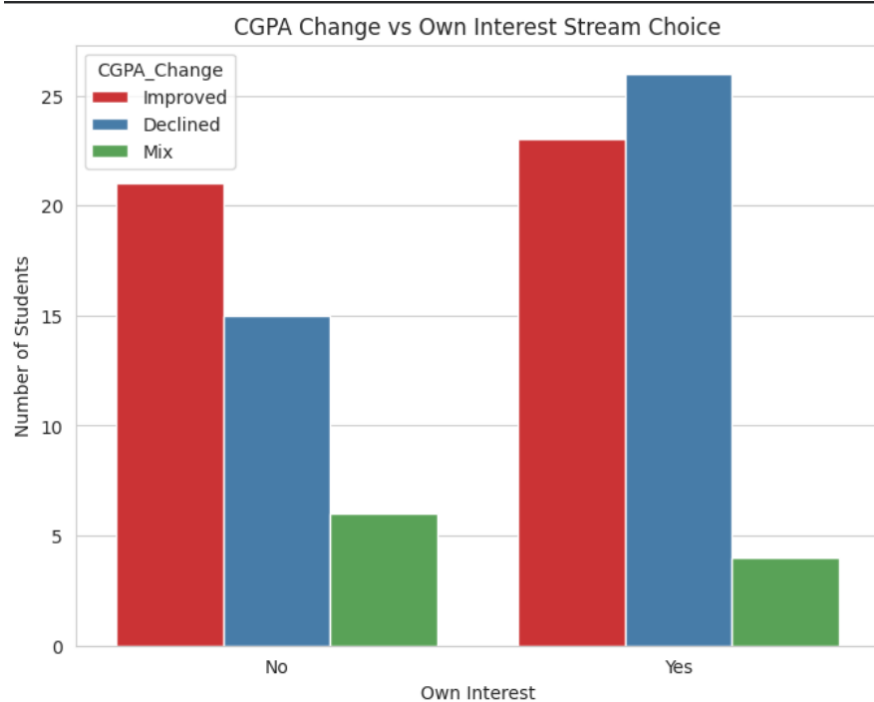
```
# CGPA_Change vs Branch
```

```
plt.figure(figsize=(12,6))
sns.countplot(x='Branch', hue='CGPA_Change', data=df, palette='Set2')
plt.title('CGPA Change Distribution by Branch')
plt.ylabel('Number of Students')
plt.xlabel('Branch')
plt.xticks(rotation=45)
plt.show()
```



CGPA_Change vs Activities / Stream Choice

```
plt.figure(figsize=(8,6))
sns.countplot(x='Own_Interest', hue='CGPA_Change', data=df, palette='Set1')
plt.title('CGPA Change vs Own Interest Stream Choice')
plt.ylabel('Number of Students')
plt.xlabel('Own Interest')
plt.xticks([0,1], ['No','Yes'])
plt.show()
```



- **Rate Our Clg**

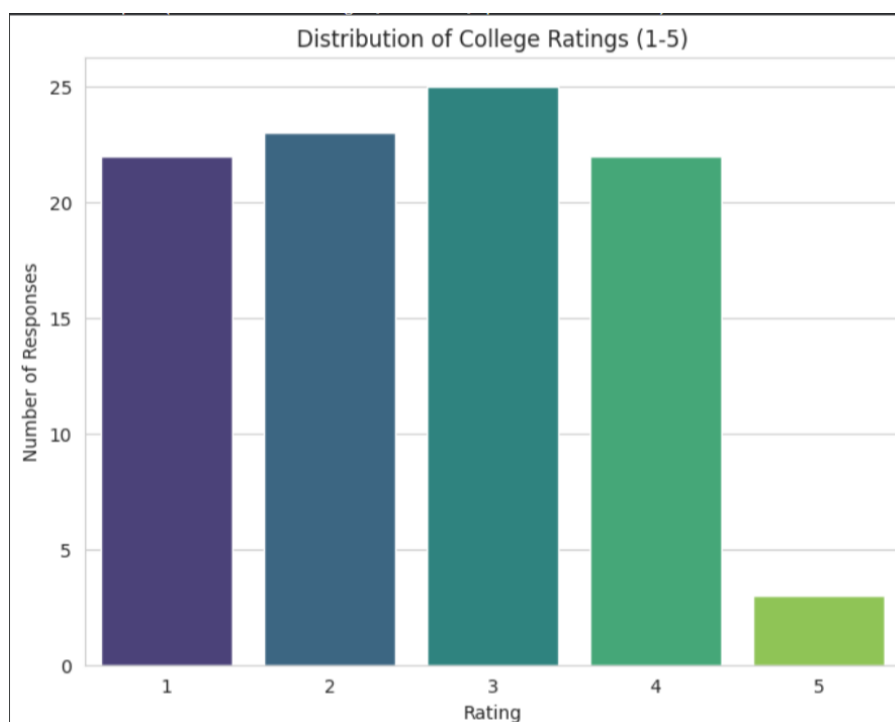
```
# Check if the column exists
if 'Rate our College' not in df.columns:
    raise ValueError("Column 'Rate our College' not found in the dataframe!")

print("Column Info:")
print(df['Rate our College'].describe())
print("\nValue Counts:")
print(df['Rate our College'].value_counts())

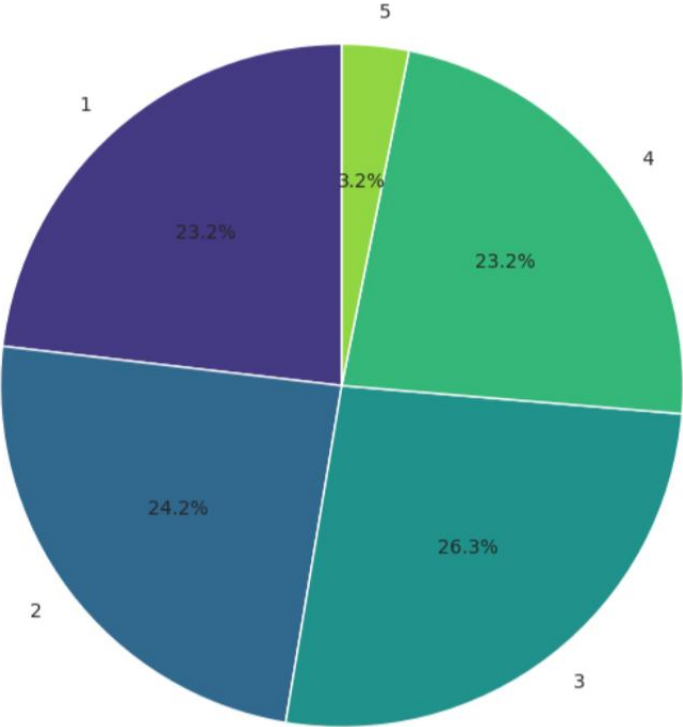
# 1: Bar Plot ---
plt.figure(figsize=(8,6))
sns.countplot(x='Rate our College', data=df, palette='viridis')
plt.title('Distribution of College Ratings (1-5)')
plt.xlabel('Rating')
plt.ylabel('Number of Responses')
plt.show()

# 2: Pie Chart ---
rating_counts = df['Rate our College'].value_counts().sort_index()
plt.figure(figsize=(8,8))
plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%', startangle=90,
        colors=sns.color_palette('viridis', 5))
plt.title('College Ratings Distribution (Pie Chart)')
plt.show()

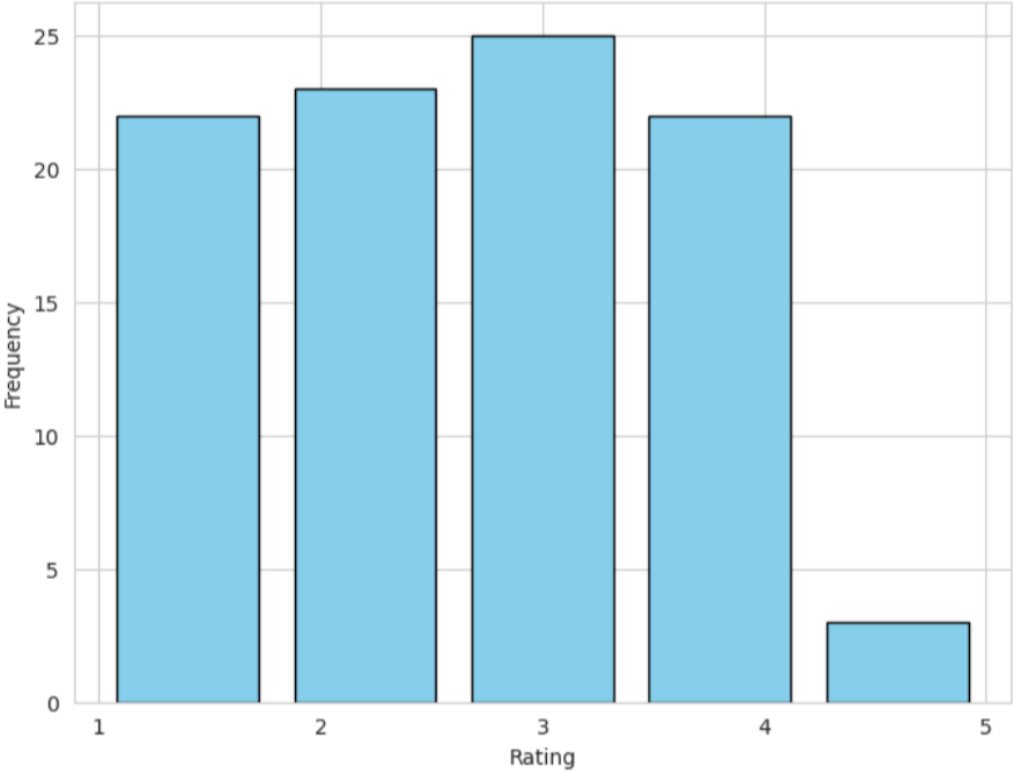
# 3: Histogram ---
plt.figure(figsize=(8,6))
plt.hist(df['Rate our College'], bins=5, color='skyblue', edgecolor='black', rwidth=0.8)
plt.title('Histogram of College Ratings')
plt.xlabel('Rating')
plt.ylabel('Frequency')
plt.xticks(range(1,6))
plt.show()
```



College Ratings Distribution (Pie Chart)



Histogram of College Ratings



- **Side Interests**

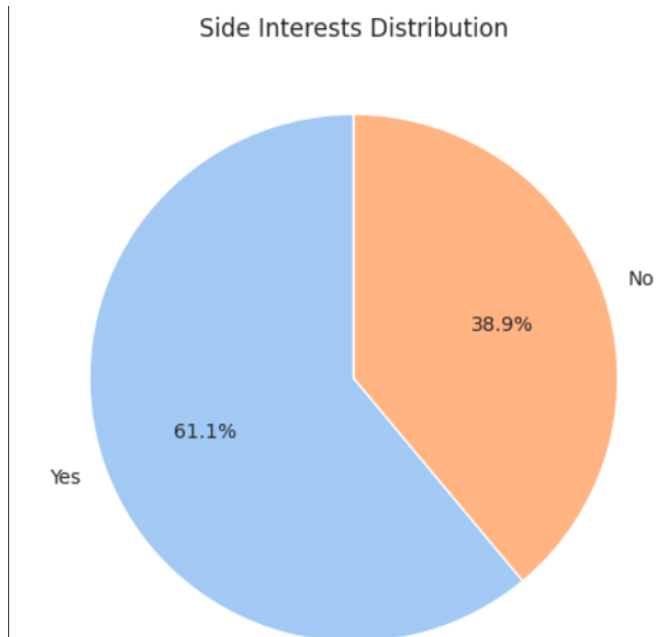
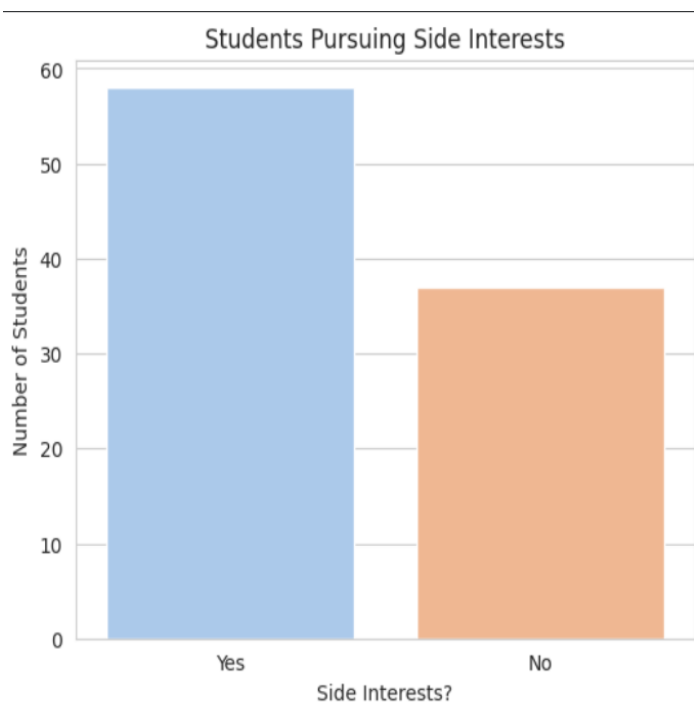
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

if 'Side_Interests' not in df.columns or 'Side_Interests_Details' not in df.columns:
    raise ValueError("Columns 'Side_Interests' and/or 'Side_Interests_Details' not found!")
print("Value Counts for Side_Interests:")
print(df['Side_Interests'].value_counts())

# 1: Bar Plot ---
plt.figure(figsize=(6,5))
sns.countplot(x='Side_Interests', data=df, palette='pastel')
plt.title('Students Pursuing Side Interests')
plt.xlabel('Side Interests?')
plt.ylabel('Number of Students')
plt.show()

# 2: Pie Chart ---
side_counts = df['Side_Interests'].value_counts()
plt.figure(figsize=(6,6))
plt.pie(side_counts, labels=side_counts.index, autopct='%1.1f%%', startangle=90,
        colors=sns.color_palette('pastel'))
plt.title('Side Interests Distribution')
plt.show()

# Show some Side_Interests_Details entries ---
yes_details = df[df['Side_Interests'] == 'Yes']['Side_Interests_Details'].dropna()
print("\nExample entries for Side_Interests_Details (students who said Yes):")
print(yes_details.head(10).to_list()) # show first 10 examples
```



```

if 'Rate our College' in df.columns:
    df.rename(columns={'Rate our College': 'Rate_College'}, inplace=True)

print("Columns in df:")
print(df.columns.tolist())

# 1. Basic Side Interests Distribution ---
plt.figure(figsize=(6,5))
sns.countplot(x='Side_Interests', data=df, palette='pastel')
plt.title('Students Pursuing Side Interests')
plt.xlabel('Side Interests?')
plt.ylabel('Number of Students')
plt.show()

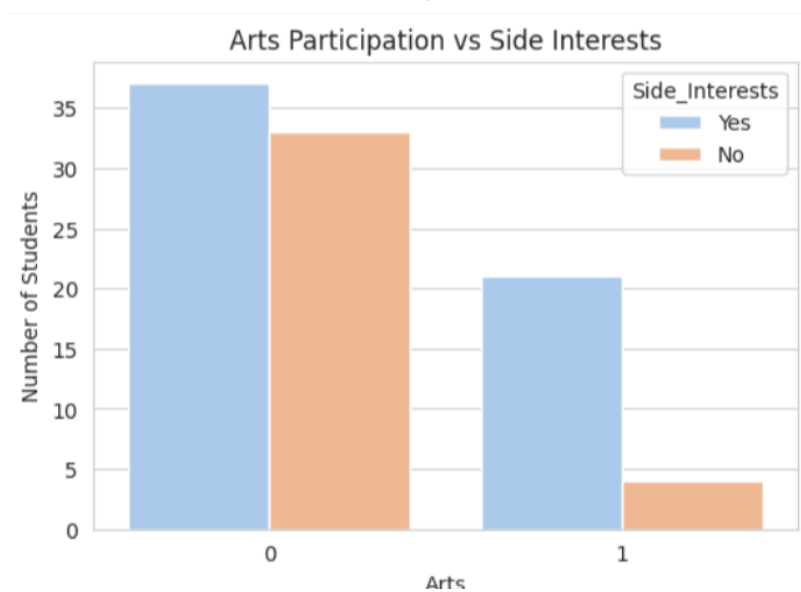
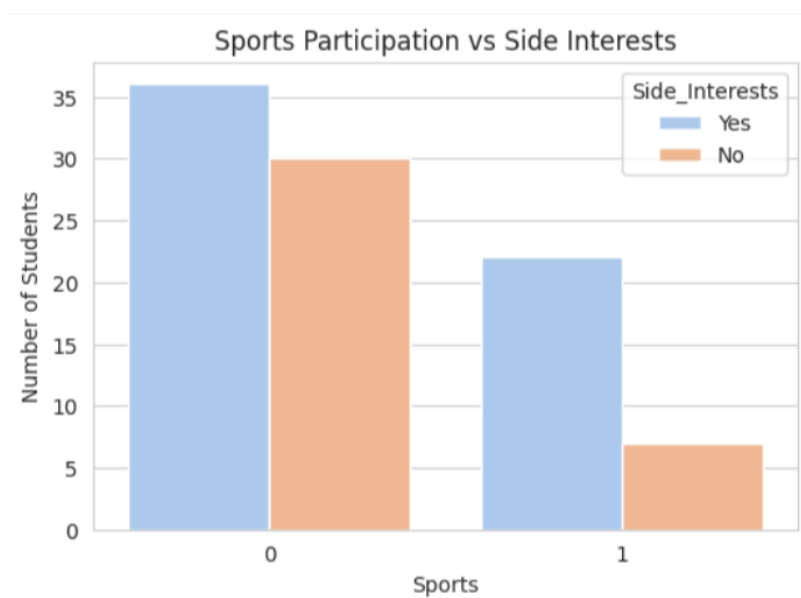
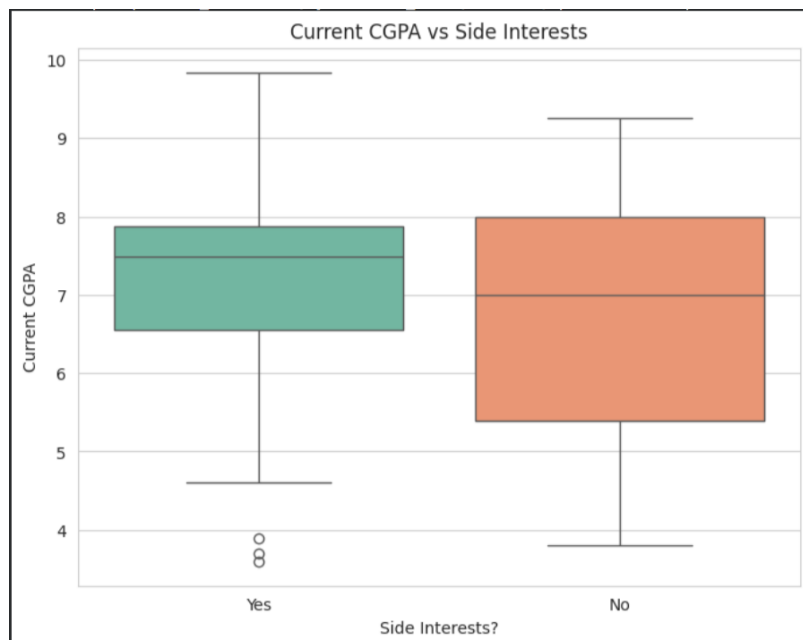
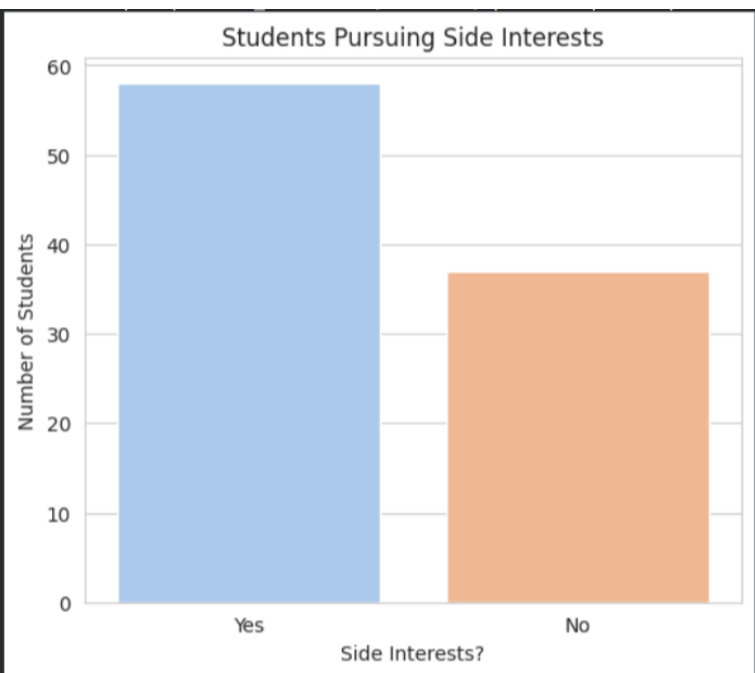
# 2. Side Interests vs Current CGPA ---
plt.figure(figsize=(8,6))
sns.boxplot(x='Side_Interests', y='Current_CGPA', data=df, palette='Set2')
plt.title('Current CGPA vs Side Interests')
plt.xlabel('Side Interests?')
plt.ylabel('Current CGPA')
plt.show()

#3. Side Interests vs Extra Activities ---
activity_cols = ['Sports', 'Arts', 'Tech', 'Academics']
for col in activity_cols:
    plt.figure(figsize=(6,4))
    sns.countplot(x=col, hue='Side_Interests', data=df, palette='pastel')
    plt.title(f'{col} Participation vs Side Interests')
    plt.xlabel(col)
    plt.ylabel('Number of Students')
    plt.show()

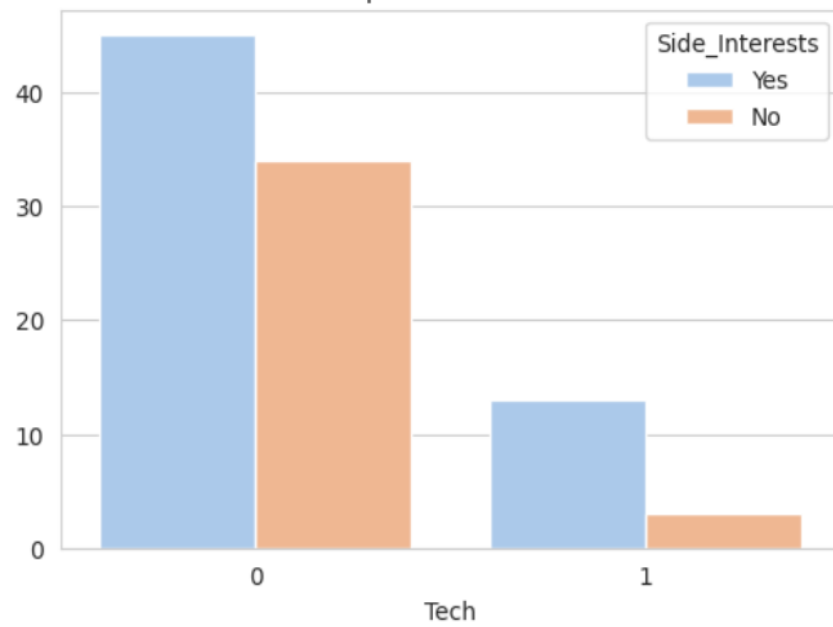
# 4. Side Interests vs Academic Challenges ---
challenge_cols = ['Interest_in_studies', 'Time_management', 'Attendance',
                  'Peer_influence', 'Health_issues', 'Lack_of_interest_in_branch', 'Other_passions']
for col in challenge_cols:
    plt.figure(figsize=(6,4))
    sns.countplot(x=col, hue='Side_Interests', data=df, palette='pastel')
    plt.title(f'{col} vs Side Interests')
    plt.xlabel(col)
    plt.ylabel('Number of Students')
    plt.xticks(rotation=45)
    plt.show()

# 5. Optional: CGPA Range vs Side Interests ---
if 'CGPA_Range' in df.columns:
    plt.figure(figsize=(8,5))
    sns.countplot(x='CGPA_Range', hue='Side_Interests', data=df, palette='Set2')
    plt.title('CGPA Range vs Side Interests')
    plt.xlabel('CGPA Range')
    plt.ylabel('Number of Students')
    plt.show()

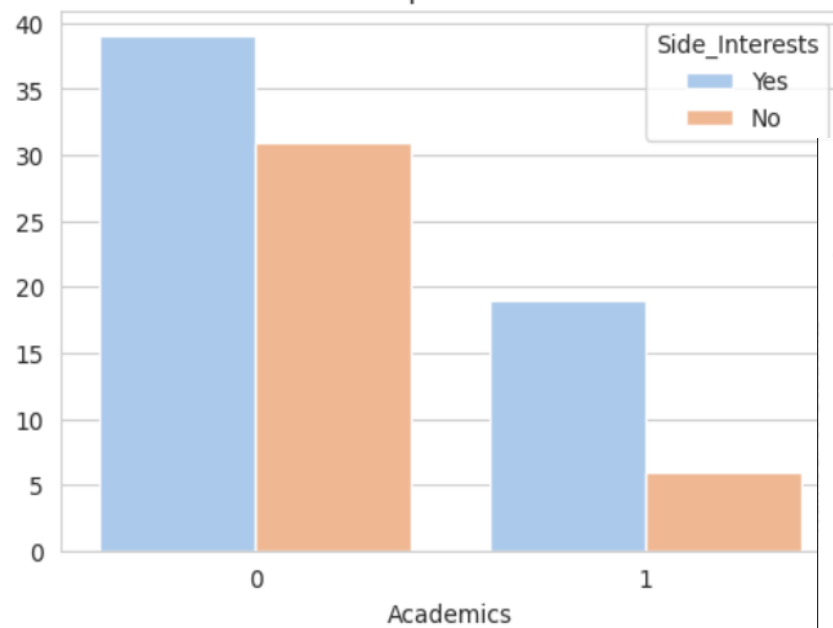
```



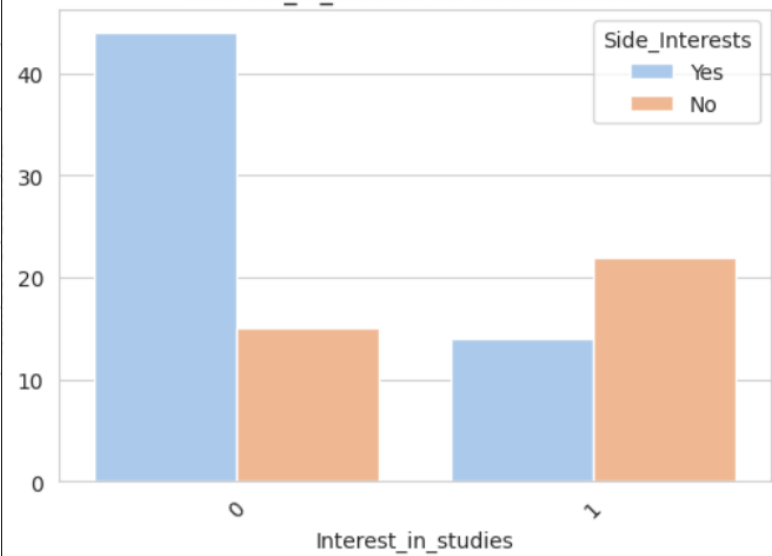
Tech Participation vs Side Interests



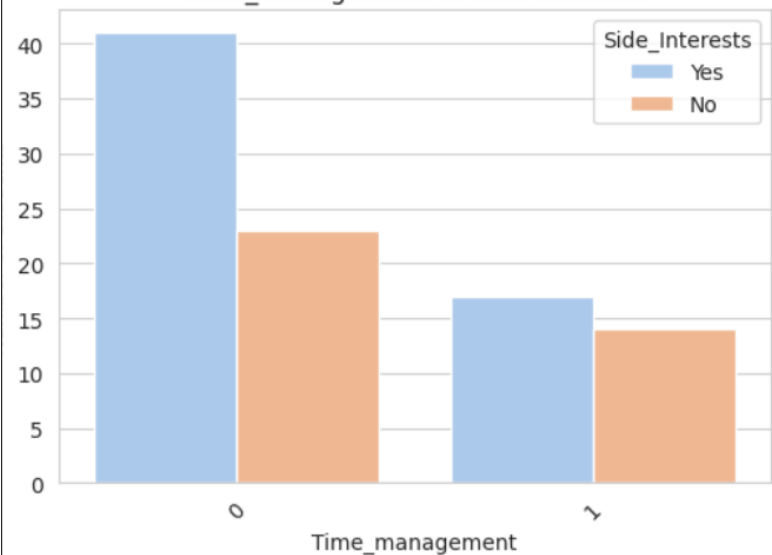
Academics Participation vs Side Interests

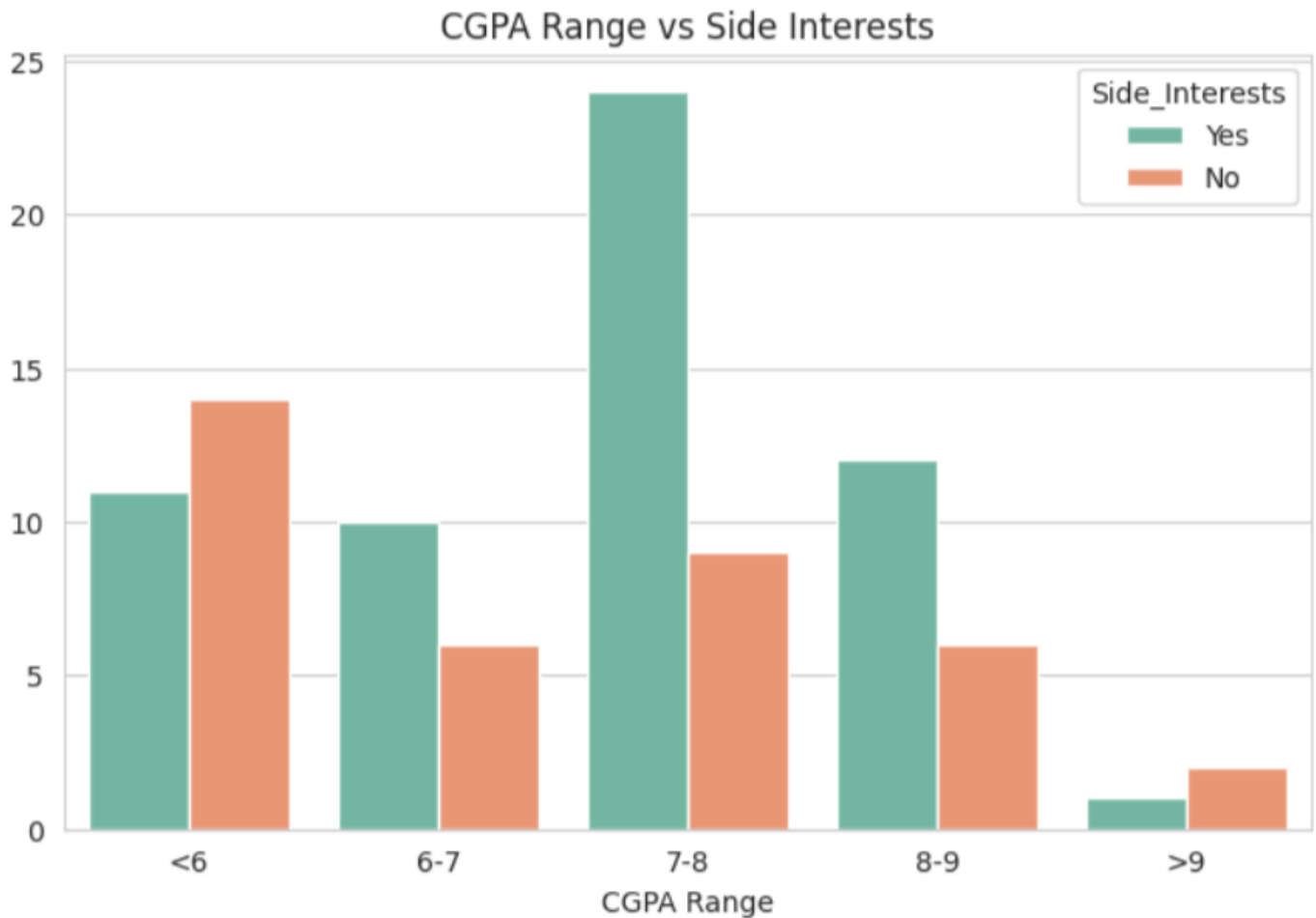


Interest_in_studies vs Side Interests



Time_management vs Side Interests





- **College Motivation**

```
# motivation_satisfaction_visualization.py

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

print("Columns in df:")
print(df.columns.tolist())

# 1. College Motivation Distribution ---
plt.figure(figsize=(7,5))
sns.countplot(x='College_Motivation', data=df, palette='Set2')
plt.title('Distribution of College Motivation')
plt.xlabel('Main Motivation for Studying')
plt.ylabel('Number of Students')
plt.xticks(rotation=30)
plt.show()

# 2. College Motivation vs Gender ---
plt.figure(figsize=(8,5))
sns.countplot(x='College_Motivation', hue='Gender', data=df, palette='pastel')
plt.title('College Motivation by Gender')
plt.xlabel('Main Motivation')
```

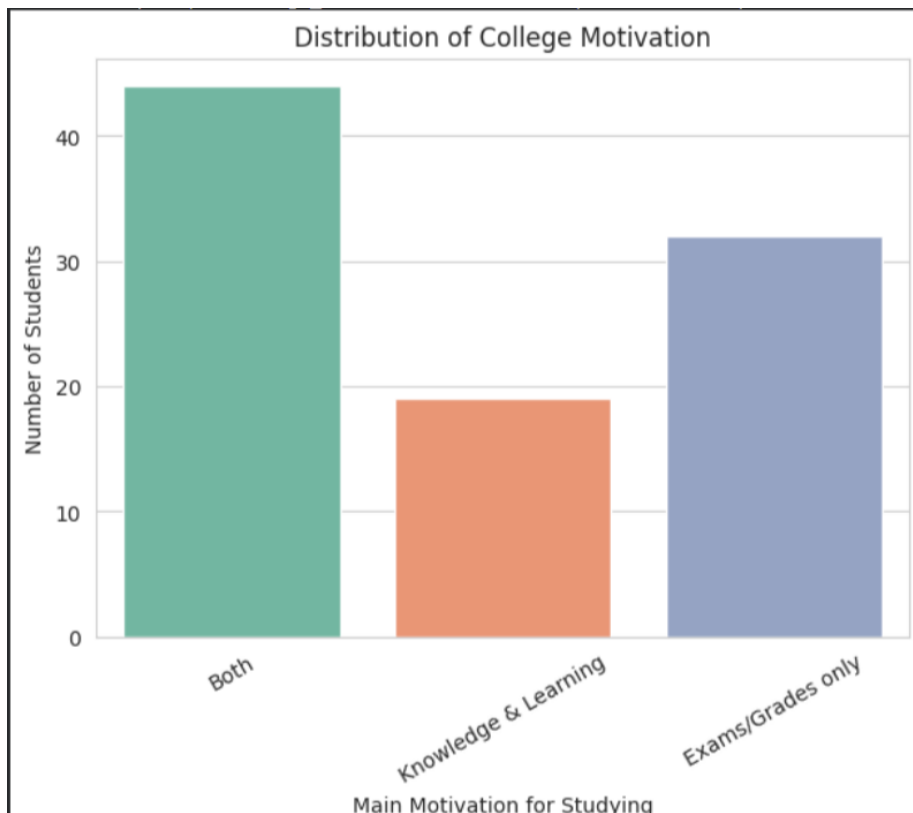
```

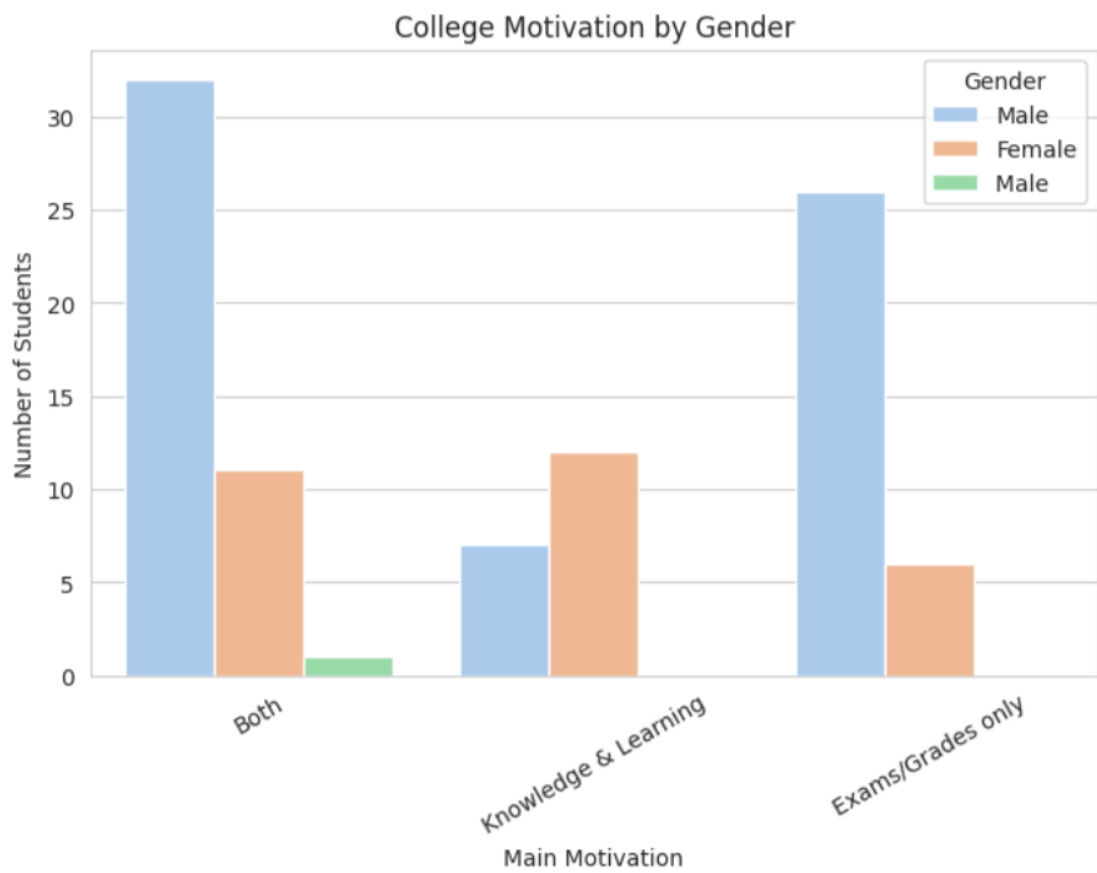
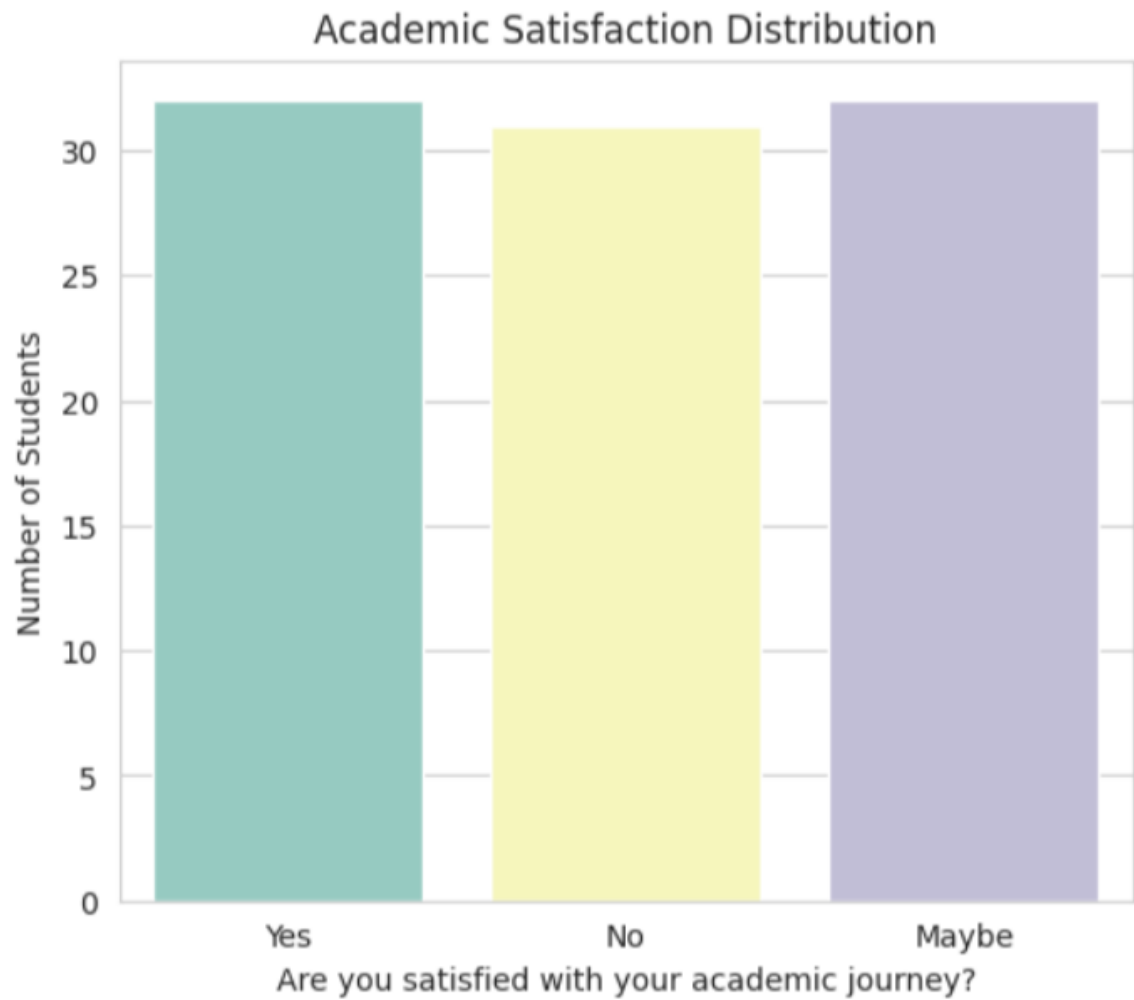
plt.ylabel('Number of Students')
plt.xticks(rotation=30)
plt.show()
# 3. Academic Satisfaction Distribution ---
plt.figure(figsize=(6,5))
sns.countplot(x='Academic_Satisfaction', data=df, palette='Set3')
plt.title('Academic Satisfaction Distribution')
plt.xlabel('Are you satisfied with your academic journey?')
plt.ylabel('Number of Students')
plt.show()

# 4. Academic Satisfaction vs Gender ---
plt.figure(figsize=(6,5))
sns.countplot(x='Academic_Satisfaction', hue='Gender', data=df, palette='pastel')
plt.title('Academic Satisfaction by Gender')
plt.xlabel('Academic Satisfaction')
plt.ylabel('Number of Students')
plt.show()

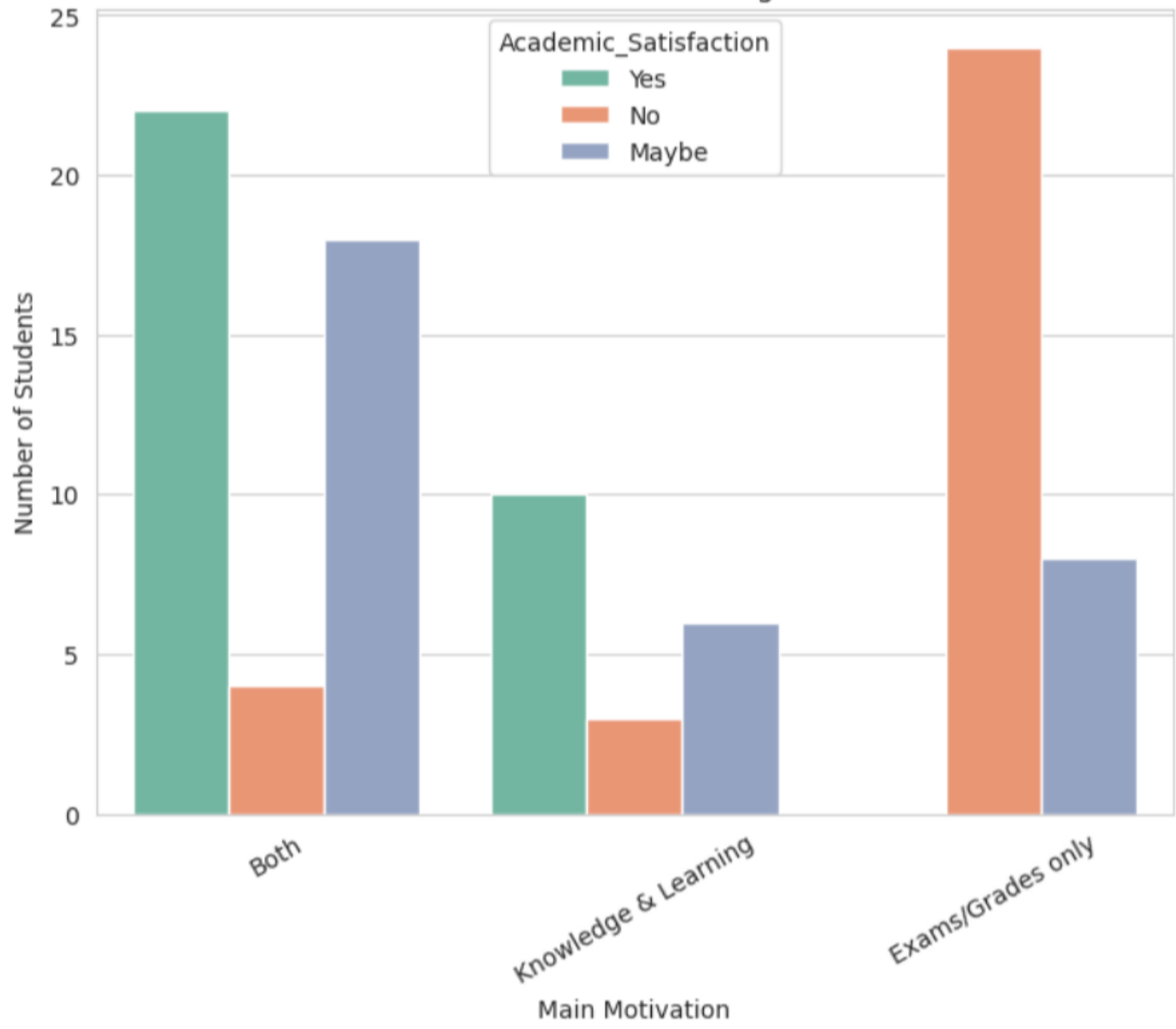
# 5. Academic Satisfaction vs College Motivation ---
plt.figure(figsize=(8,6))
sns.countplot(x='College_Motivation', hue='Academic_Satisfaction', data=df, palette='Set2')
plt.title('Academic Satisfaction vs College Motivation')
plt.xlabel('Main Motivation')
plt.ylabel('Number of Students')
plt.xticks(rotation=30)
plt.show()

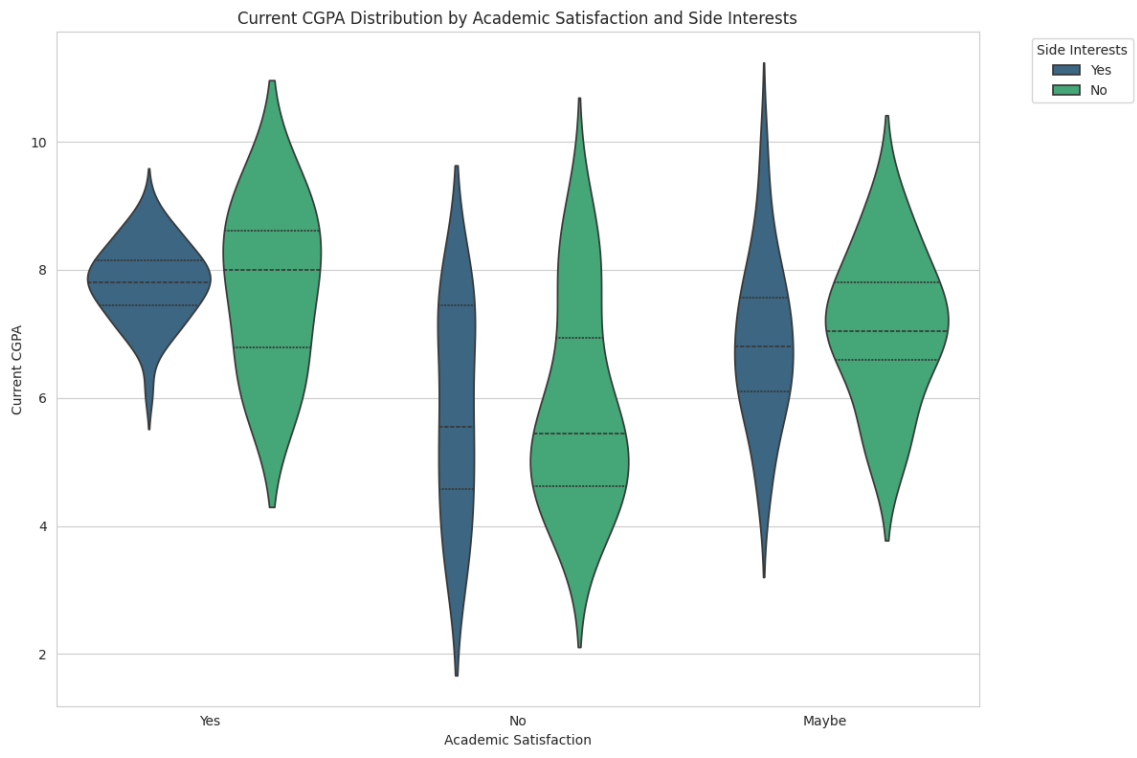
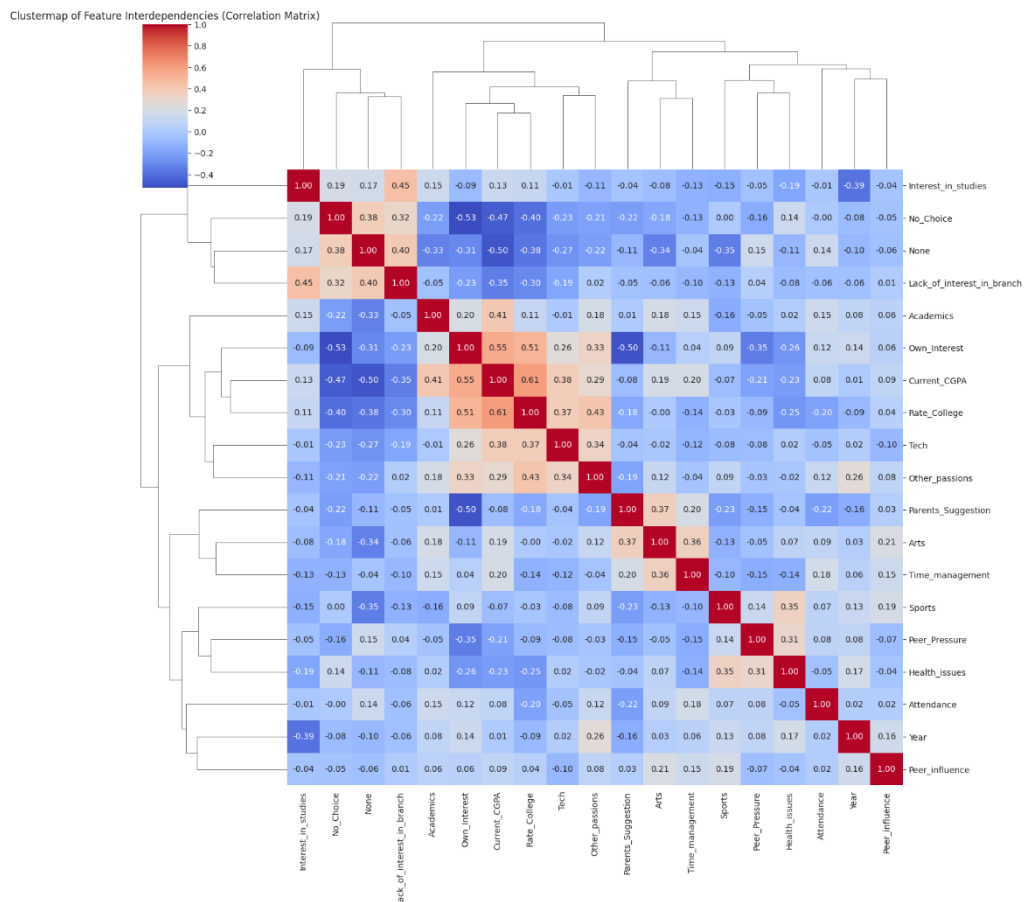
```



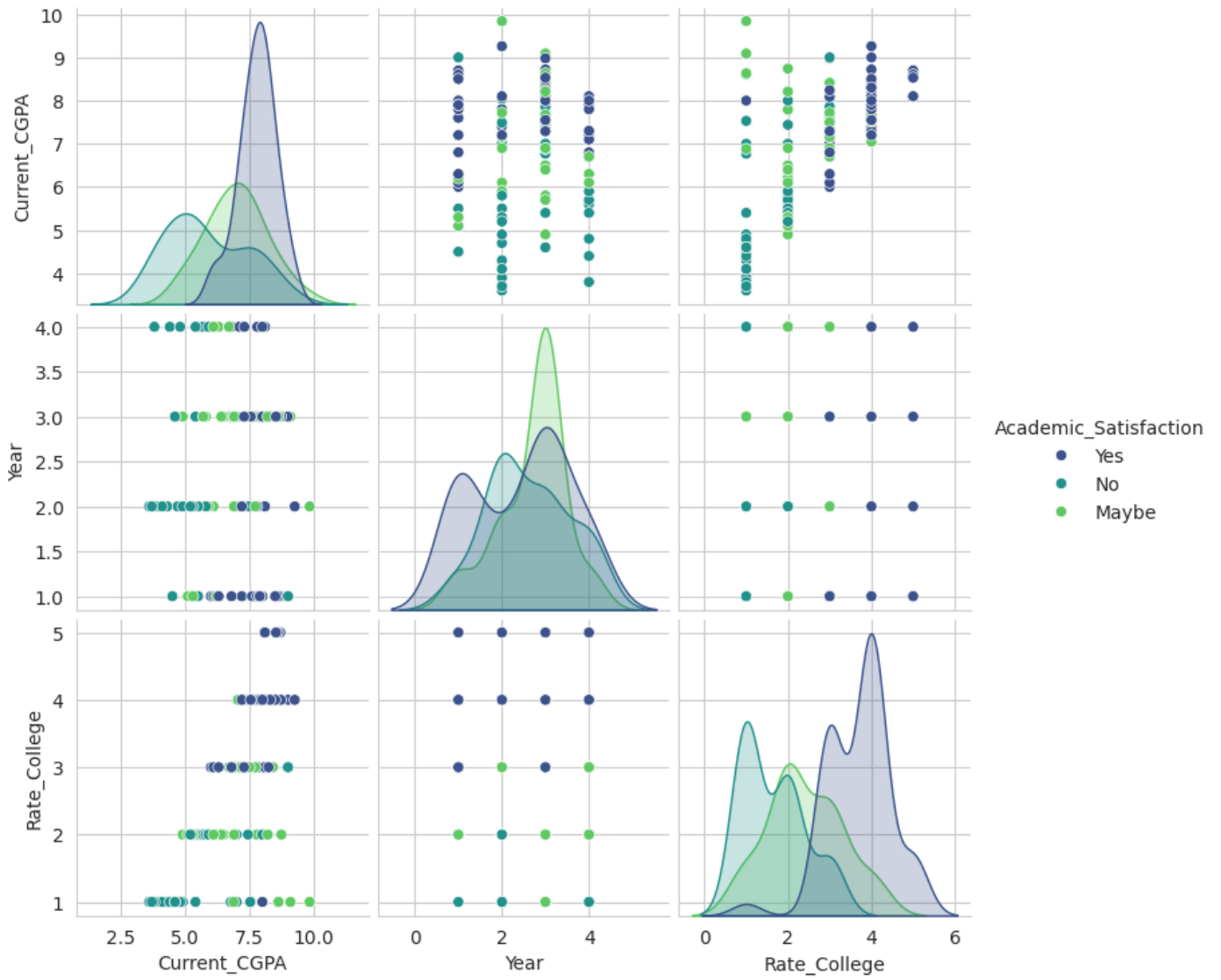


Academic Satisfaction vs College Motivation

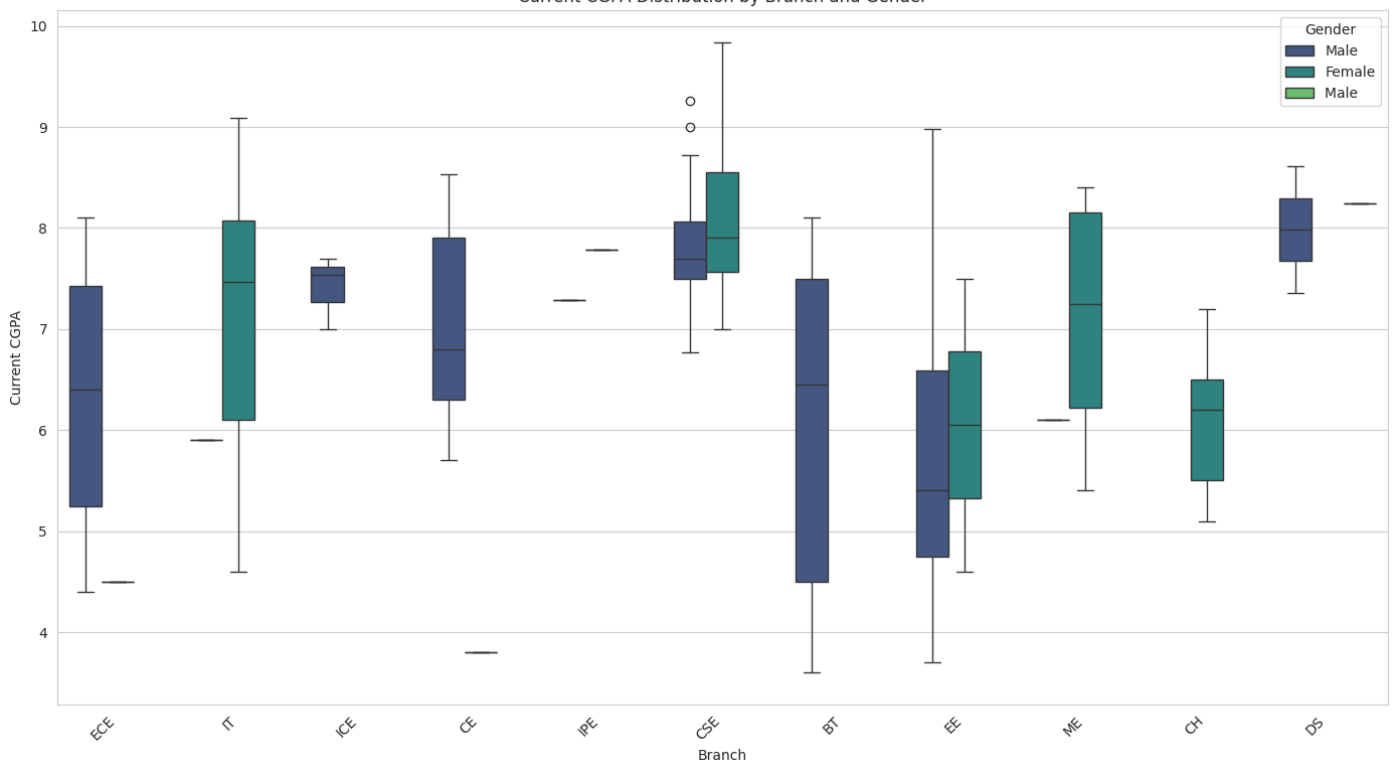




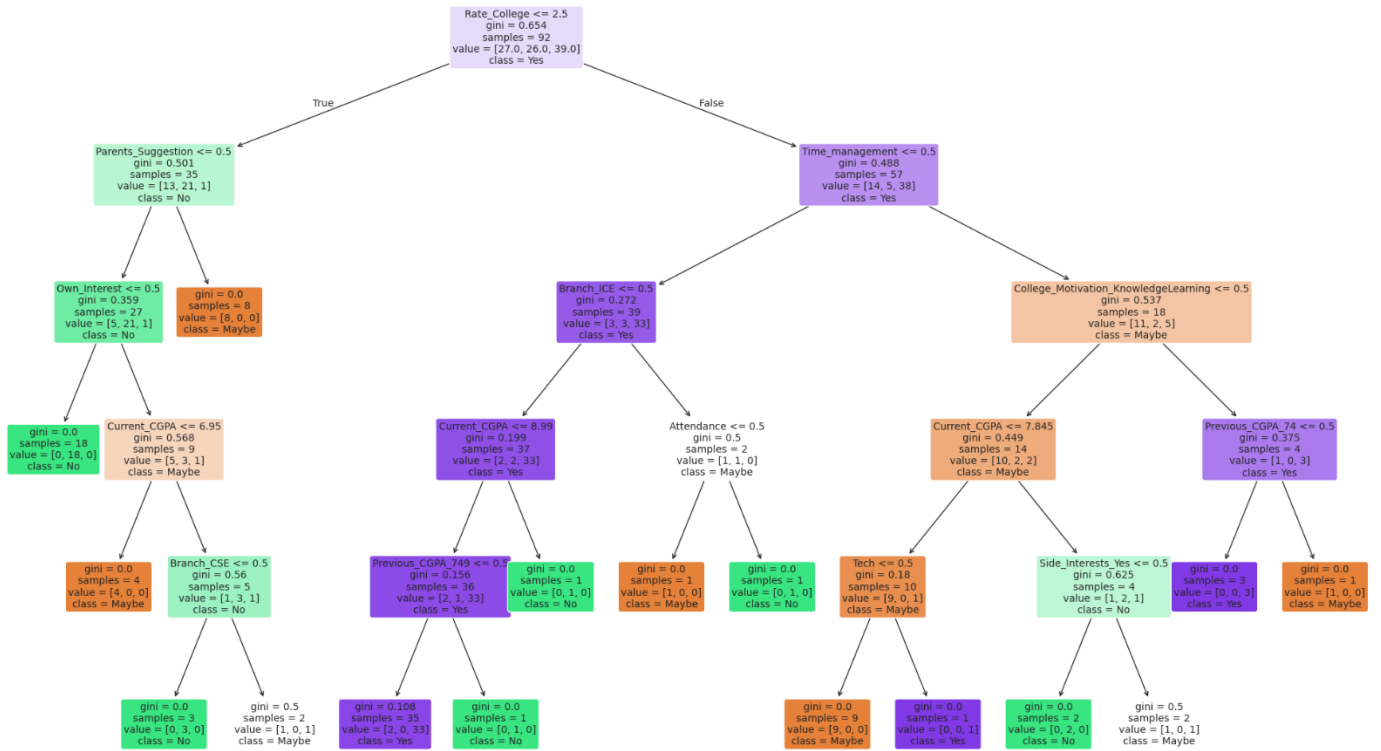
Pair Plot of Academic Metrics by Academic Satisfaction



Current CGPA Distribution by Branch and Gender



Decision Tree for Academic Satisfaction Prediction



END